HO CHI MINH CITY UNIVERSITY OF

TECHNOLOGY AND ENGINEERING

FACULTY OF ADVANCED TRAINING



FINAL REPORT

COURSE: EMBEDDED SYSTEMS

TOPIC:

BUILDING EMBEDDED LINUX WITH YOCTO ON RASPBERRY PI 4, DEPLOYING HARDWARE-ACCELERATED EDGE AI AND DEVELOPING A DEVICE DRIVER

GROUP 4

Supervisor:	M.Sc. Huỳnh Hoàng Hà	
Students:	Đỗ Gia Bảo	Student ID: 23161046
	Tô Quốc Hiệp	Student ID: 22161050
	Nguyễn Đoàn Hoàng Ngân	Student ID: 24119064
	Đặng Nguyễn Nam Nhật	Student ID: 22119022
	Tô Ngọc Thảo Uyên	Student ID: 24119098

Ho Chi Minh City, October 2026

HO CHI MINH CITY UNIVERSITY OF

TECHNOLOGY AND ENGINEERING

FACULTY OF ADVANCED TRAINING



FINAL REPORT

COURSE: EMBEDDED SYSTEMS

TOPIC:

BUILDING EMBEDDED LINUX WITH YOCTO ON RASPBERRY PI 4, DEPLOYING HARDWARE-ACCELERATED EDGE AI AND DEVELOPING A DEVICE DRIVER

GROUP 4

Supervisor: M.Sc. Huỳnh Hoàng Hà

Students:	Đỗ Gia Bảo	Student ID: 23161046
	Tô Quốc Hiệp	Student ID: 22161050
	Nguyễn Đoàn Hoàng Ngân	Student ID: 24119064
	Đặng Nguyễn Nam Nhật	Student ID: 22119022
	Tô Ngọc Thảo Uyên	Student ID: 24119098

Ho Chi Minh City, October 2026

PROJECT ASSIGNMENT FORM

SOCIALIST REPUBLIC OF VIETNAM
Independence – Freedom – Happiness

Ho Chi Minh City, October 12, 2026

Group: GROUP 4 **Course:** EMBEDDED SYSTEMS

No.	Student name	Student ID	Role
1	Đỗ Gia Bảo	23161046	Virtualization, QEMU emulation
2	Tô Quốc Hiệp	22161050	Embedded co-developer
3	Nguyễn Đoàn Hoàng Ngân	24119064	Device driver development, report writing
4	Đặng Nguyễn Nam Nhật	22119022	Team leader
5	Tô Ngọc Thảo Uyên	24119098	Device driver development

Assignment date: August 18, 2026 **Report submission date:** December 13, 2026

1. Topic: BUILDING EMBEDDED LINUX WITH YOCTO ON RASPBERRY PI 4, DEPLOYING HARDWARE-ACCELERATED EDGE AI AND DEVELOPING A DEVICE DRIVER.

2. Initial materials provided by the supervisor:

. . . . . .

3. Project content:

. . . . . .

. . . . . .

4. Final deliverables:

. . . . . .

PROGRAM DIRECTOR
(Signature and full name)

SUPERVISOR
(Signature and full name)

SUPERVISOR'S COMMENTS

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Score: . . . . . .

Ho Chi Minh City, … / … / 2026

Supervisor

(Signature and full name)

TABLE OF CONTENTS

PROJECT ASSIGNMENT FORM . . . . . **i**

SUPERVISOR'S COMMENTS . . . . . **ii**

TABLE OF CONTENTS . . . . . **iii**

LIST OF ABBREVIATIONS . . . . . **v**

LIST OF FIGURES . . . . . **vi**

LIST OF TABLES . . . . . **vii**

CHAPTER 1 Overview **1**

1.1 Problem Statement 1

1.2 Objectives and Scope 2

1.3 Task Assignment and Report Structure 2

CHAPTER 2 Theoretical Background **4**

2.1 Embedded Linux and the Yocto Project 4

2.1.1 Embedded Linux and cross-compilation 4

2.1.2 Basic concepts of Yocto 5

2.1.3 BitBake and the build process 6

2.2 Raspberry Pi 4 and the Boot Process 8

2.3 Edge Deep Learning and C versus Python 10

2.4 Linux Device Drivers and QEMU 12

CHAPTER 3 System Design and Implementation **15**

3.1 Building Embedded Linux with Yocto on the Raspberry Pi 4 15

3.1.1 Configuration, compilation and characteristics of the Yocto image 15

3.1.2 Image inspection, SD card flashing and UART debugging 17

3.2 Deploying the Edge AI System on the Raspberry Pi 4 19

3.2.1 The `meta-edge-ai` layer and the TensorFlow Lite packages . . . 20

3.2.2 The MobileNet V1 model and input data 20

3.2.3 Two inference engines: Python and native C 22

3.3 Developing a Device Driver on the QEMU Emulation Platform 22

3.3.1 Environment, Yocto image and the `meta-mydriver` layer 23

3.3.2 Design and implementation of the `mychardev` driver 26

3.3.3 Recipe, image integration and running on QEMU 31

CHAPTER 4 Experimental Results and Evaluation **35**

4.1 Edge AI Inference Performance on the Raspberry Pi 4 35
4.2 Bottlenecks, CPU Governor and Accuracy 38
4.3 QEMU Emulation Results and Driver Testing 39
4.4 Overall Evaluation 43

CHAPTER 5 Conclusion and Future Work 45
5.1 Conclusion 45
5.2 Limitations and Future Work 46

REFERENCES 47

APPENDIX A Linux Command Reference Table 49

APPENDIX B Source Code of the `mychardev` Driver 53
B.1 Driver source code `mychardev.c` 53
B.2 Makefile 54
B.3 Recipe `mychardev_0.1.bb` 55

LIST OF ABBREVIATIONS

Abbreviation	Full term
AI	Artificial Intelligence
ALU	Arithmetic Logic Unit
API	Application Programming Interface
ARM	Advanced RISC Machines (ARM processor architecture)
BSP	Board Support Package
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DTB	Device Tree Blob (binary device tree file)
EEPROM	Electrically Erasable Programmable ROM
GPIO	General-Purpose Input/Output
GPL	GNU General Public License
GPU	Graphics Processing Unit
I/O	Input/Output
KVM	Kernel-based Virtual Machine
LPDDR4	Low-Power Double Data Rate 4 (4th-generation low-power DDR memory)
LTS	Long Term Support
OE	OpenEmbedded (OpenEmbedded build system)
QEMU	Quick Emulator
RAM	Random Access Memory
RSS	Resident Set Size
RPM	RPM Package Manager (software package format)
SoC	System on Chip
SSH	Secure Shell (secure remote access protocol)
sstate	Shared State Cache
TFLite	TensorFlow Lite (TensorFlow Lite inference library)
UART	Universal Asynchronous Receiver/Transmitter
VFS	Virtual File System
WIC	OpenEmbedded Image Creator (Yocto’s disk image creation tool)

LIST OF FIGURES

Figure 2.1 Layered architecture of an embedded Linux system 4
Figure 2.2 Standard task chain when BitBake processes a recipe 7
Figure 2.3 Overall build flow of the Yocto Project 8
Figure 2.4 Flowchart of the 7-stage boot process of the Raspberry Pi 4 9
Figure 2.5 Path of a system call from the application to the driver 13

Figure 3.1 Flowchart of building and deploying Yocto Linux for the Raspberry Pi 4 16
Figure 3.2 Yocto layers of the Raspberry Pi 4 project 20
Figure 3.3 Edge AI software stack on the Raspberry Pi 4 21
Figure 3.4 The four processing stages of each image and their average times (native C, 1 thread) 22
Figure 3.5 Structure of the project directory 24
Figure 3.6 Structure of the `meta-mydriver` layer 25
Figure 3.7 Lifecycle flowchart of the `mychardev` driver 27
Figure 3.8 Flowchart of the `mychardev_read()` function 29
Figure 3.9 Flowchart of the `mychardev_write()` function 29
Figure 3.10 Path of the driver from source code to the QEMU virtual machine . . 32
Figure 3.11 Architecture of the QEMU emulation environment 34

Figure 4.1 Mean inference latency by number of threads (lower is better) 36
Figure 4.2 Total time to process 1,000 images (lower is better) 37
Figure 4.3 Throughput when processing 1,000 images (the bold horizontal line is the real-time threshold of 20 images/s) 37
Figure 4.4 Peak resident set size (RSS): native C needs only 30.4% of Python's memory 37

LIST OF TABLES

Table 1.1 Team organization and task assignment 3

Table 2.1 Basic terms in the Yocto Project 5
Table 2.2 Comparison of BitBake and GNU Make 6
Table 2.3 Subsystems of the BCM2711 SoC on the Raspberry Pi 4 Model B . . . 8
Table 2.4 Boot files and their architectural roles 11
Table 2.5 Comparison of conventional character device registration and misc devices 14

Table 3.1 Experimental environment of the Edge AI part 16
Table 3.2 Meaning of the configuration variables 18
Table 3.3 Inspection results of the partitions of the WIC image 18
Table 3.4 Verification results on the Raspberry Pi 4 board 19
Table 3.5 Results of building Yocto Linux for the Raspberry Pi 4 19
Table 3.6 Edge AI packages verified on the target machine 21
Table 3.7 Configuration of the development machine (host) 23
Table 3.8 Meaning of the configuration variables in `local.conf` 24
Table 3.9 Explanation of the components of the recipe `mychardev_0.1.bb` . . 32
Table 3.10 BitBake tasks executed for the `mychardev` recipe 33
Table 3.11 Parameters of the QEMU virtual machine generated by `runqemu` . . . 34

Table 4.1 Comparison of Python and native C for single-image inference with 1 thread 35
Table 4.2 Mean latency by number of CPU threads 36
Table 4.3 Results of the 1,000-image benchmark 36
Table 4.4 Breakdown of the per-image processing time by stage 38
Table 4.5 Comparison of the `ondemand` and `performance` governors 39
Table 4.6 Classification accuracy for each of the 10 Imagenette classes 39
Table 4.7 Summary of the Edge AI metrics 40
Table 4.8 Main output files of the build process 40
Table 4.9 Test cases for the `mychardev` driver 42
Table 4.10 Objectives versus results 44

Table A.1 Reference table of the Linux commands used in the project 49

CHAPTER 1

OVERVIEW

1.1 Problem Statement

The growth of edge computing and the Internet of Things (IoT) has fundamentally changed modern embedded systems. Traditional microcontrollers often lack the memory capacity, computing power and software ecosystem needed to run complex algorithms such as convolutional neural networks (CNNs). Conversely, full-featured desktop operating systems (such as Ubuntu Desktop or Debian) bring a large baseline resource overhead, unpredictable scheduling latency and a storage footprint that is hard to control.

Embedded Linux offers the optimal balance: it leverages the networking capabilities, driver ecosystem and multi-core scheduling of the Linux kernel, while being modular enough to remove every unnecessary component. Through deterministic build frameworks such as the Yocto Project, engineers can create an operating system that contains exactly the drivers, system libraries and executables the problem requires, thereby maximizing reliability, security and hardware utilization.

Constraints of embedded systems

Deploying deep neural networks on edge hardware poses several challenges:

Memory scarcity: embedded platforms usually have limited RAM. High-level prototyping languages such as Python consume significant memory for interpreter state, dynamic type management and the garbage collector.

Execution latency: edge devices often have to process sensor streams (e.g., computer vision frames) under hard or soft real-time latency thresholds (above 20 frames per second).

Thermal and power limits: edge processors operate within a limited thermal envelope. Overclocking or locking the cores at maximum frequency can cause thermal throttling or degrade the hardware.

Software abstraction versus native implementation: in research and prototyping, deep learning models are often treated as a "black box". The instructor's requirement is to view the neural network as a purely deterministic mathematical program, and to compare natively compiled code (C) with an interpreted environment (Python).

1.2 Objectives and Scope

To meet the instructor's evaluation criteria, the project investigates four core research questions:

Question 1: how can a customized minimal Linux distribution be cross-compiled from source, deterministically, with Yocto, so that it supports both low-level hardware debugging and an Edge AI runtime on the Raspberry Pi 4?

Question 2: what exactly is the physical and software boot architecture of the BCM2711 SoC on the Raspberry Pi 4, and how can boot time be measured and optimized systematically?

Question 3: what is the quantitative trade-off (inference latency, execution jitter, peak RSS memory consumption) between a native C inference engine and the Python runtime?

Question 4: can thread parallelism across multiple CPU cores accelerate the neural network to real-time throughput (above 20 fps) without pruning or retraining the model, and what is the main bottleneck of the system when running at scale?

In addition to the four questions above, following the instructor's supplementary direction after the model deployment milestone, the group pursued two further objectives:

Set up a **QEMU emulation environment** within the Yocto framework, allowing the system to be built, run and tested directly on the development machine without real hardware.

Research, design and implement a **device driver** as a kernel module, integrate it into the system image with a Yocto recipe, and test it on QEMU.

Scope: the Edge AI part is deployed and measured on a real Raspberry Pi 4 Model B board; the device driver part is built and tested only on the QEMU emulation platform (target machine `qemux86-64`) and is not deployed to real hardware.

1.3 Task Assignment and Report Structure

The roles and tasks of the members are summarized in Table 1.1.

Report structure

The report consists of 5 chapters:

Chapter 1 – Overview: problem statement, constraints of embedded systems, research questions, objectives, scope and task assignment.

Chapter 2 – Theoretical Background: embedded Linux and the Yocto Project,

Table 1.1: Team organization and task assignment

No.	Full name	Role	Assigned technical tasks
1	Đặng Nguyễn Nam Nhật	Team leader (project architect)	Planned the project, managed the schedule and procured the hardware (Raspberry Pi 4 Model B kit, accessories, USB-UART adapter); supervised all technical phases; implemented the Edge AI subsystem: the Yocto layer `meta-edge-ai`, TensorFlow Lite integration, export of the C API headers, the native C and Python inference engines, multi-thread optimization, the 1,000-image benchmark and the performance analysis.
2	Tô Quốc Hiệp	Embedded co-developer	Independently reproduced the Yocto build pipeline in parallel (Poky Scarthgap and `meta-raspberrypi`) to verify reproducibility in a constrained host environment (50 GB disk limit, using `rm_work`); researched and wrote the formal architectural documentation of the 7-stage boot flow of the Raspberry Pi 4.
3	Nguyễn Đoàn Hoàng Ngân, Tô Ngọc Thảo Uyên	Device driver development (kernel subsystem); report writer (Hoàng Ngân)	Researched, designed and implemented a custom Linux device driver (a supplementary task assigned by the instructor after the model deployment milestone); Nguyễn Đoàn Hoàng Ngân also compiled the final report.
4	Đỗ Gia Bảo	Virtualization specialist	Researched, installed and configured the QEMU target emulation environment within the Yocto framework.

the Raspberry Pi 4 and its 7-stage boot flow, edge deep learning with TensorFlow Lite, C versus Python, Linux device drivers and QEMU.

Chapter 3 – System Design and Implementation: building embedded Linux with Yocto on the Raspberry Pi 4, deploying the Edge AI system, and developing a device driver on the QEMU emulation platform.

Chapter 4 – Experimental Results and Evaluation: inference performance (single image, multi-core, 1,000 images), bottlenecks, the CPU governor, accuracy, QEMU emulation results and driver testing.

Chapter 5 – Conclusion and Future Work.

CHAPTER 2

THEORETICAL BACKGROUND

2.1 Embedded Linux and the Yocto Project

2.1.1 Embedded Linux and cross-compilation

A typical embedded Linux system consists of four main components, brought up from the bottom up (Figure 2.1):

Cross toolchain: the compiler, linker and C library that run on the development machine (host) but generate code for the target machine (target).

Bootloader: initializes the basic hardware and loads the kernel into memory (e.g., U-Boot, GRUB). On QEMU, the kernel is loaded directly with the `-kernel` option.

Linux kernel: manages processes, memory, file systems, networking and device drivers.

Root filesystem: contains the init process (`init`), libraries, command-line utilities, configuration files and applications.

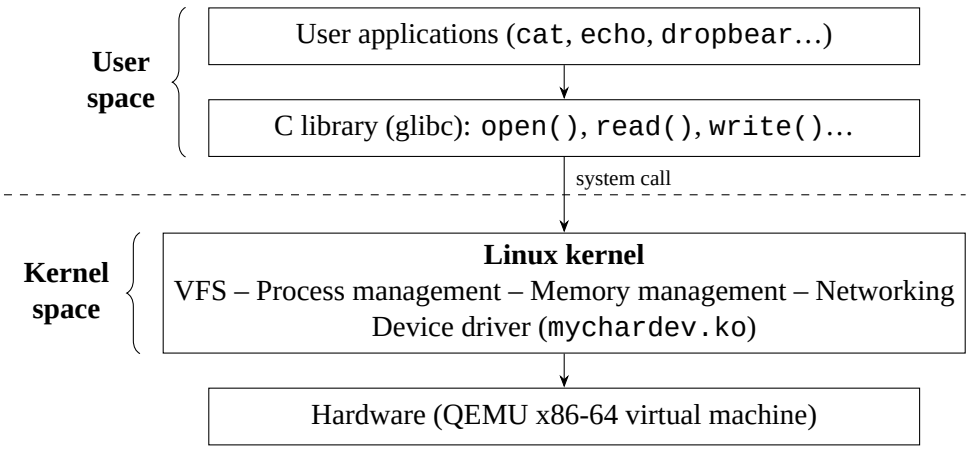


Figure 2.1: Layered architecture of an embedded Linux system

Cross-compilation

Cross-compilation is the process of compiling source code on one platform (the host, here an x86-64 computer running Ubuntu) to produce executable code for another platform (the target). In this project, the real target is the Raspberry Pi 4 (ARM architecture), while the emulated QEMU target is x86-64. Even when host and target share the

x86-64 architecture, Yocto still uses its own toolchain (`x86_64-poky-linux-gcc`) together with an independent sysroot. As a result, the build output is completely independent of the host operating system's libraries, which guarantees reproducibility; and switching between targets (e.g., from `qemux86-64` to `raspberrypi4`) only requires changing the `MACHINE` variable.

2.1.2 Basic concepts of Yocto

The Yocto Project is an open-source project under the Linux Foundation that provides tools, processes and metadata for creating customized Linux distributions for embedded systems, independent of the hardware architecture. Yocto **is not** a Linux distribution, but a tool **for creating** Linux distributions.

Yocto publishes two releases per year (April and October), each with its own code name. Long-term support (LTS) releases are maintained for 4 years. This project uses **Scarthgap 5.0** (LTS, released in April 2024).

Table 2.1 summarizes the important Yocto terms.

Table 2.1: Basic terms in the Yocto Project

Term	Meaning
Poky	Yocto's reference distribution, including BitBake, OE-Core and sample configuration
OpenEmbedded	The build system and core metadata set (OE-Core, layer `meta`) used by Yocto
BitBake	Task execution tool that reads metadata and orchestrates the build process
Recipe (`.bb`)	File describing how to build a software package
Append (`.bbappend`)	File that extends/modifies an existing recipe without editing the original file
Class (`.bbclass`)	Logic shared by many recipes, loaded with `inherit`
Layer	Directory grouping related recipes, classes and configuration (naming convention `meta-*`)
MACHINE	Describes the target hardware (e.g., `qemux86-64`, `raspberrypi4`)
DISTRO	Distribution policy (name, version, init system, features)
Image	Special recipe that assembles packages into a complete root filesystem
sstate-cache	Cache storing task results for reuse across builds
BSP layer	Layer supporting a specific hardware family (e.g., `meta-raspberrypi` for the Raspberry Pi)
rm_work	Class that deletes a recipe's work directory after it has been built, reducing disk usage from over 80 GB to under 30 GB

Poky – the reference distribution

Poky is Yocto's "starter kit", containing every component needed to build a complete Linux system. The main directory structure of Poky consists of:

`bitbake/`: the BitBake build tool.

`meta/`: the OE-Core core layer with about a thousand basic recipes (gcc, glibc, busybox, linux-yocto, openssl…).

`meta-poky/`: configuration of the `poky` distro.

`meta-yocto-bsp/`: support for several reference boards.

`meta-skeleton/`: sample recipes (e.g., `hello-mod` – a kernel module template).

`scripts/`: helper tools such as `runqemu`, `devtool`, `wic`, `bitbake-layers`.

`oe-init-build-env`: script that initializes the environment and the build directory.

2.1.3 BitBake and the build process

BitBake is a task execution tool written in Python, playing a role similar to `make` but at the scale of an entire operating system. When the command `bitbake <target>` is run, BitBake proceeds as follows:

1. **Parse:** reads the configuration files (`bitbake.conf`, `local.conf`, `bblayers.conf`, `layer.conf`) and all recipes, appends and classes in the layers.
2. **Build the dependency graph:** determines all tasks to be performed and the dependencies between them.
3. **Check the cache:** each task has a signature (hash) computed from its input data; if the signature matches a result in sstate-cache, BitBake reuses the existing result (`_setscene` task) instead of executing it again.
4. **Parallel execution:** runs independent tasks concurrently (limited by `BB_NUMBER_THREADS`).

Table 2.2 compares BitBake with the traditional `make` tool.

Table 2.2: Comparison of BitBake and GNU Make

Criterion	**GNU Make**	**BitBake**
Input	A single `Makefile`	Thousands of recipes from many layers
Change detection	Compares file modification times	Compares content and configuration signatures (hashes)
Scope	One project	Hundreds of projects combined into one operating system
Cache	None	sstate-cache shared across builds and machines

Recipes and the task lifecycle

A recipe is a `.bb` file describing how to build a software package. File names follow the convention `<name>_<version>.bb`, from which BitBake derives the variables `PN` (package name) and `PV` (package version). A recipe answers the questions: where to get the source code (`SRC_URI`), what the license is (`LICENSE`, `LIC_FILES_CHKSUM`), what is needed at build time (`DEPENDS`), what is needed at run time (`RDEPENDS`), and how to compile and install it (`do_compile`, `do_install`).

Each recipe is processed through the standard task chain shown in Figure 2.2.

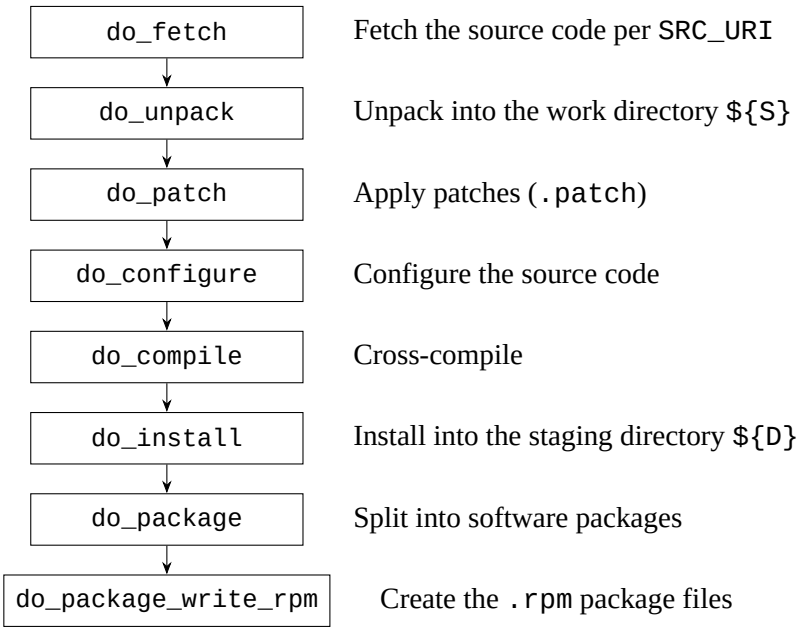


Figure 2.2: Standard task chain when BitBake processes a recipe

For an image recipe, BitBake additionally performs the `do_rootfs` task (installing all packages listed in `IMAGE_INSTALL` into the root filesystem) and `do_image_*` (packaging into formats such as `.ext4`, `.tar.bz2`).

To avoid repeating logic, a recipe can **inherit classes** with the `inherit` keyword. Some typical classes: `autotools`, `cmake`, `meson` for common build systems; `module` for kernel modules; `core-image` for image recipes.

Layer

A layer is Yocto's mechanism for organizing metadata modularly. Each layer is a directory containing a `conf/layer.conf` file that declares the layer name, the recipe search path pattern, the priority and the compatible Yocto version. A layer is activated by adding its path to the `BBLAYERS` variable in `bblayers.conf`. An important principle is **never to modify third-party layers directly** (such as `meta`); all customizations are placed in your own layer through new recipes or `.bbappend` files.

Overall build flow

Figure 2.3 describes the overall data flow of a Yocto build.

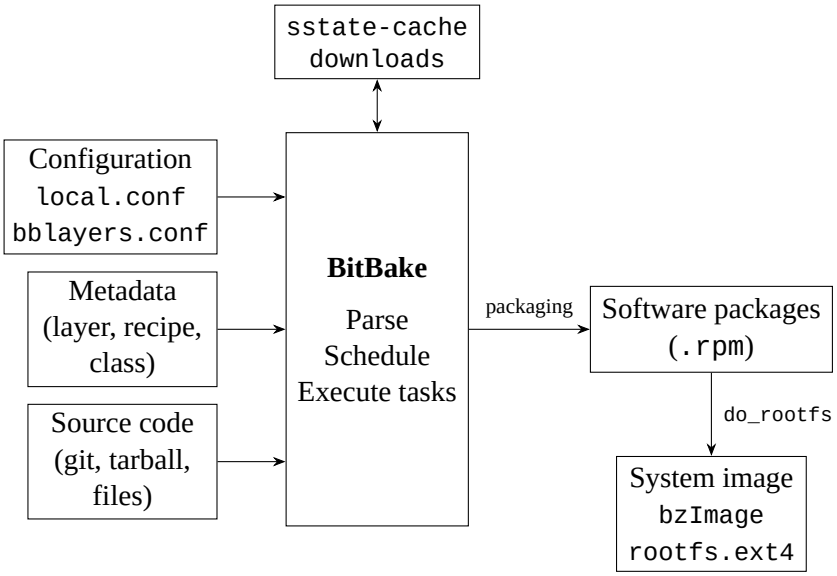


Figure 2.3: Overall build flow of the Yocto Project

2.2 Raspberry Pi 4 and the Boot Process

Hardware architecture of the BCM2711 SoC

The Raspberry Pi 4 Model B is built around the Broadcom BCM2711 SoC, manufactured on a high-performance 28 nm process. Its hardware subsystems are summarized in Table 2.3.

Table 2.3: Subsystems of the BCM2711 SoC on the Raspberry Pi 4 Model B

Component	Characteristics
CPU cluster	4 ARMv8-A 64-bit Cortex-A72 cores at a base clock of 1.5 GHz. Each core has a 48 KB L1 instruction cache and a 32 KB L1 data cache, and the cores share a 1 MB L2 cache. The Cortex-A72 architecture supports out-of-order superscalar execution and the NEON SIMD processor.
GPU subsystem	VideoCore VI at 500 MHz, supporting OpenGL ES 3.0 and H.265/HEVC 4Kp60 hardware video decoding. Notably, the VideoCore GPU acts as the master controller when the board is powered on.
Memory	4 GB of high-speed LPDDR4 SDRAM on a 32-bit bus, providing large memory bandwidth for the matrix multiplications of neural networks.
Peripherals	Dedicated EMMC2 SD card controller, two USB 3.0 controllers, Gigabit Ethernet and hardware UARTs (PL011 and Mini-UART interfaces).

The 7-stage boot process

On a typical x86 personal computer, the central processing unit (CPU) is the first to execute when power is applied: the CPU loads the BIOS or UEFI firmware from SPI flash memory to enumerate the hardware and find the bootloader (e.g., GRUB). In contrast, the hardware architecture of the Raspberry Pi follows a **GPU-first boot** model.

On the BCM2711 SoC, the ARM Cortex-A72 cores are initially held in sleep (hard reset) when 5 V power is applied. The VideoCore VI GPU acts as the main system controller: it executes the early boot code, initializes the hardware power rails, configures the LPDDR4 memory, loads the Linux kernel into RAM and only then releases the CPU reset signal. The entire process is shown in Figure 2.4.

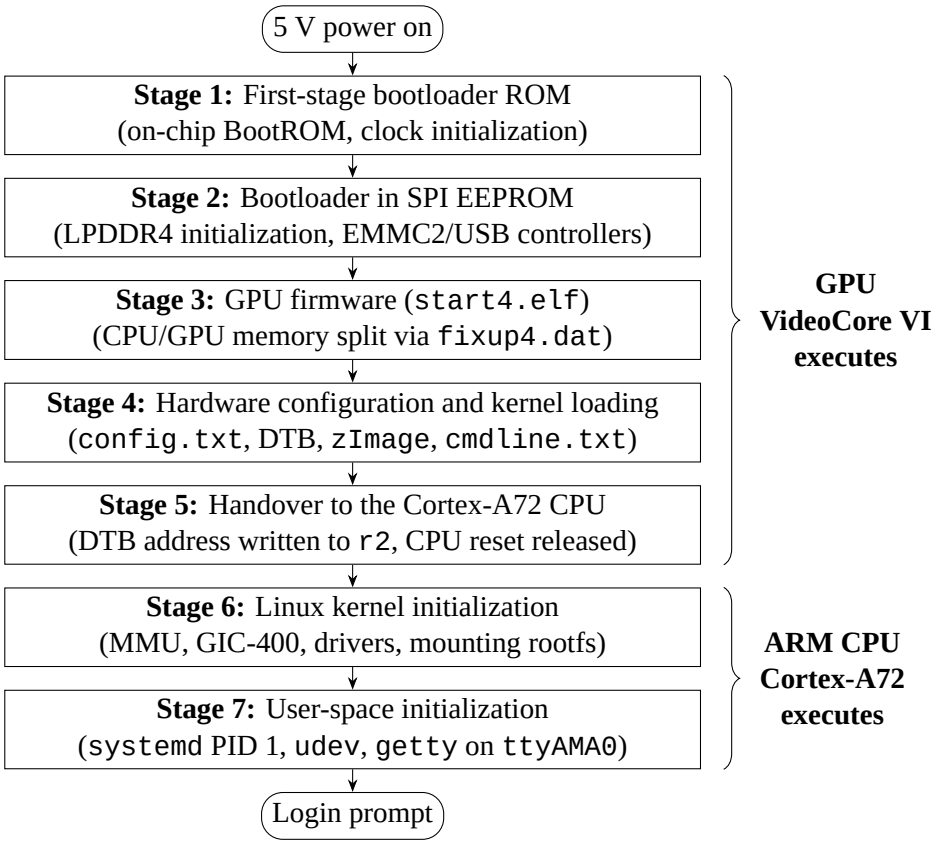


Figure 2.4: Flowchart of the 7-stage boot process of the Raspberry Pi 4

The details of each stage are as follows:

Stage 1 – First-stage bootloader ROM: when 5 V DC power is applied, the VideoCore VI GPU core is powered while the ARM cores remain in reset. The GPU executes a minimal, immutable bootloader etched into on-chip mask ROM (BootROM). The BootROM initializes the basic clocks and searches for the second-stage bootloader.

Stage 2 – Second-stage bootloader in SPI EEPROM: on older generations (Raspberry Pi 1, 2, 3), the BootROM looked for the file `bootcode.bin` on the FAT32 partition of the SD card. The Raspberry Pi 4 fundamentally changes this architecture by

mounting a 4 MB SPI EEPROM chip directly on the board. The BootROM loads the bootloader code from this EEPROM. The EEPROM bootloader initializes the LPDDR4 SDRAM controller and establishes communication with the storage device (SD card via EMMC2, or the USB controller).

Stage 3 – The GPU executes the `start4.elf` firmware: once the LPDDR4 SDRAM is accessible, the GPU reads the first FAT32 partition of the MicroSD card. The GPU loads the main GPU operating system firmware `start4.elf` together with the memory management coordination file `fixup4.dat`. The `fixup4.dat` file determines how memory is split: how much of the 4 GB of LPDDR4 is given to the ARM CPU and how much to the VideoCore GPU.

Stage 4 – Hardware configuration and kernel loading: the GPU firmware parses `config.txt` and applies low-level hardware parameters such as `enable_uart=1`, clock frequencies and HDMI mode. The GPU then loads into RAM the device tree blob (`bcm2711-rpi-4-b.dtb`) together with the active overlays, loads the compressed Linux kernel (`zImage`), and reads the kernel boot arguments from `cmdline.txt`.

Stage 5 – Handover to the ARM Cortex-A72 cluster: the GPU writes the physical address of the loaded Device Tree Blob into register `r2` of the ARM CPU core, points the program counter to the kernel entry point in RAM and releases the reset signal of the Cortex-A72 cluster. The ARM CPU cores wake up and take control of the board.

Stage 6 – Linux kernel initialization and driver binding: the ARM CPU starts executing `zImage`. The kernel decompresses itself, configures the MMU and the GIC-400 interrupt controller, parses the device tree to initialize the drivers, and reads `cmdline.txt` to mount the root filesystem from partition 2 (`root=/dev/mmcblk0p2`, ext4).

Stage 7 – User-space initialization (systemd, PID = 1): the kernel launches the first user process with PID 1 at `/sbin/init`. In the group's Yocto build, `/sbin/init` is a symbolic link pointing directly to `/lib/systemd/systemd`. Systemd mounts the virtual file systems (`/proc`, `/sys`), runs udev, starts the system services and spawns a serial `getty` process on `/dev/ttyAMA0`, producing an interactive login prompt.

The role of each file in the boot process is summarized in Table 2.4.

2.3 Edge Deep Learning and C versus Python

Edge deep learning and TensorFlow Lite

In advanced academic theory, neural networks are often pictured through complex biological metaphors. From an embedded systems engineering perspective, however, a

Table 2.4: Boot files and their architectural roles

File	Location	Executed by	Function and role
EEPROM code	On-board SPI flash	VideoCore VI GPU	Initializes the LPDDR4 memory controller and the EMMC2/USB controllers
`start4.elf`	Partition 1 (FAT32)	VideoCore VI GPU	Main GPU operating system; loads the device tree, overlays and kernel
`fixup4.dat`	Partition 1 (FAT32)	`start4.elf`	Configures how the memory address space is split between the ARM CPU and the GPU
`config.txt`	Partition 1 (FAT32)	`start4.elf`	Hardware directives (UART enable, CPU clock, display parameters)
`cmdline.txt`	Partition 1 (FAT32)	Linux kernel	Kernel arguments: serial console (`ttyAMA0`), root partition, log level
`bcm2711-rpi-4-b.dtb`	Partition 1 (FAT32)	Linux kernel	Binary device tree describing the BCM2711 hardware address map
`zImage`	Partition 1 (FAT32)	ARM Cortex-A72	Compressed 32-bit ARM Linux kernel executed by the CPU
`/lib/systemd/systemd`	Partition 2 (ext4)	ARM Cortex-A72	PID 1 init system: initializes services, file systems and the serial `getty`

neural network is essentially a **deterministic mathematical function**: a directed acyclic graph of tensor operations, consisting of multi-dimensional matrix multiplications, additions, nonlinear activation functions (ReLU) and spatial pooling.

Google's TensorFlow Lite (TFLite) is an open-source deep learning framework designed specifically for on-device inference. TFLite models are stored as immutable FlatBuffers (`model.tflite`), which can be executed directly from a memory-mapped file without unpacking.

A key technique of embedded AI is **post-training quantization**. The standard weights and activations of a neural network are represented as 32-bit floating-point numbers (`float32`). Quantization converts these parameters into 8-bit unsigned integers (`uint8`) through an affine transformation:

$$\text{Real value} = \text{Scale} \times (\text{Quantized value} - \text{Zero_Point}) \tag{2.1}$$

By mapping continuous values onto 256 discrete integer levels $[0, 255]$, the model size is reduced by 75% (from about 16 MB to 4.1 MB for MobileNet V1). More importantly, the integer arithmetic logic units (ALUs) of the ARM Cortex-A72 cores compute integer dot products considerably faster than floating-point operations, while consuming less energy.

Natively compiled C versus interpreted Python

Python provides an intuitive environment for rapid prototyping, but brings significant overhead on resource-constrained embedded systems:

Memory overhead: Python must load the CPython virtual machine, dynamic symbol tables and runtime libraries (e.g., NumPy), consuming more than 30 MiB of RAM before executing any statement. A C executable links only the shared libraries it needs, keeping its footprint around 10 MiB.

Deterministic execution: Python's automatic garbage collector periodically pauses the program to traverse the object graph and reclaim memory. In real-time embedded systems, this causes non-deterministic latency spikes. Native C uses explicit memory management (`malloc`/`free` or static buffers).

Deployment footprint: a C program cross-compiled by Yocto runs as a standalone ELF file, requiring no Python interpreter, packaging framework or `site-packages` in the root filesystem.

2.4 Linux Device Drivers and QEMU

Concepts and classification

A device driver is code that runs in kernel space and provides a standard interface through which user-space applications access a device via familiar system calls. Linux divides drivers into three main types:

Character device: accessed as a byte stream, without block buffering (e.g., serial ports, sensors, GPIO). This is the type implemented in this project.

Block device: accessed in addressable blocks of data (disks, memory cards).

Network device: sends and receives packets; does not appear as a file in `/dev`.

Each character or block device is identified by a pair of numbers **(major, minor)**: the major number identifies the driver, while the minor number distinguishes the devices managed by the same driver.

Kernel modules

A kernel module (`.ko`) is a piece of code that can be loaded into (`insmod`, `modprobe`) or removed from (`rmmod`) the kernel while the system is running, without recompiling the kernel or rebooting. Each module has two entry points: an initialization function registered with `module_init()` and a cleanup function registered with `module_exit()`.

A module can be built *in-tree* (inside the kernel source tree) or *out-of-tree* (outside

it, compiled against an already configured kernel). An out-of-tree module must be compiled against exactly the version and configuration of the running kernel; otherwise, the kernel refuses to load it with an *invalid module format* error.

The `file_operations` structure and the VFS

When an application calls `read()` on the file `/dev/mychardev`, the kernel's virtual file system (VFS) layer looks up the `struct file_operations` attached to that device and calls the driver's corresponding `.read` function (Figure 2.5). This is the core mechanism that allows every device in Linux to be accessed uniformly "as a file".

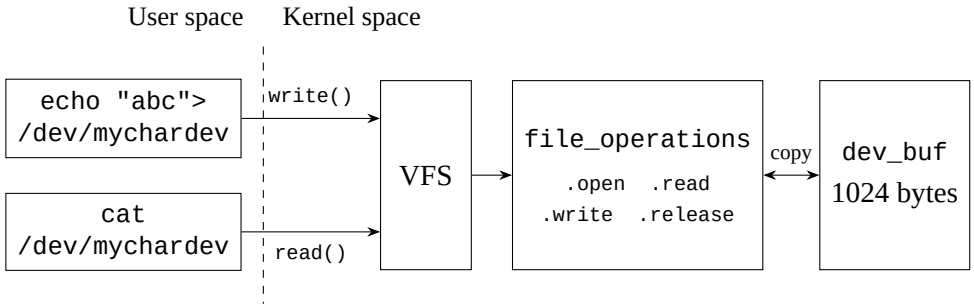


Figure 2.5: Path of a system call from the application to the driver

Because the memory of user processes and kernel memory are separated, the driver must not directly dereference pointers passed in by the user (marked `__user`); it must instead use the functions `copy_to_user()` and `copy_from_user()`, or helper functions that wrap them such as `simple_read_from_buffer()` and `simple_write_to_buffer()`.

Misc devices

To register a character device the conventional way, a driver must request a device number (`alloc_chrdev_region`), initialize and add a `cdev` (`cdev_init`, `cdev_add`), and create a class and a device (`class_create`, `device_create`) so that the file in `/dev` is created automatically. The Linux kernel provides the **misc device** class to simplify this whole process: all misc devices share major number 10, and a single call to `misc_register()` registers the device and automatically creates the `/dev/<name>` node (Table 2.5).

Access synchronization

Several processes may read from or write to a device at the same time. To protect the shared data (the buffer and the data length), the driver uses a **mutex** lock: only one process may enter the critical section at a time, while the other processes sleep until the lock is released.

Table 2.5: Comparison of conventional character device registration and misc devices

Conventional approach	**Misc device**
`alloc_chrdev_region()` requests the major/minor numbers	Shares major 10, minor allocated dynamically
`cdev_init()` + `cdev_add()`	A single call to `misc_register()`
`class_create()` + `device_create()`	The `/dev` node is created automatically
Removal through several corresponding functions	A single call to `misc_deregister()`

The QEMU emulator

QEMU (Quick Emulator) is an open-source emulator and virtualizer capable of emulating many CPU architectures (x86, ARM, RISC-V, MIPS…) together with peripheral devices. When the virtual machine's architecture matches the host, QEMU can be combined with **KVM** to execute code directly on the real CPU, reaching speeds close to real hardware.

Yocto provides the `runqemu` script, which boots a freshly built image without the need to type QEMU's long command-line parameters by hand. The script reads the `*.qemuboot.conf` file generated together with the image to determine the kernel path, the root filesystem and the virtual machine configuration. Some important options:

`nographic`: does not open a graphical window; the serial console is shown directly in the terminal.

`slirp`: user-mode networking, requiring no administrator privileges; host port 2222 is forwarded to port 22 (SSH) of the virtual machine.

`kvm`: enables hardware virtualization acceleration.

CHAPTER 3

SYSTEM DESIGN AND IMPLEMENTATION

This chapter presents the three work streams of the project: building embedded Linux with Yocto on the Raspberry Pi 4 (Section 3.1), deploying the Edge AI system on top of that operating system (Section 3.2) and developing a device driver on the QEMU emulation platform (Section 3.3).

3.1 Building Embedded Linux with Yocto on the Raspberry Pi 4

This section presents the entire process of building a customized embedded Linux distribution from source with the Yocto Project for the Raspberry Pi 4 Model B board: preparing the host machine, configuring and compiling the image, inspecting the image with a loop device, flashing the SD card, booting the board and debugging over the UART port. This is the operating system foundation for running the Edge AI system in Section 3.2, and it answers Research Questions 1 and 2 posed in Chapter 1.

3.1.1 Configuration, compilation and characteristics of the Yocto image

The development environment was set up on an x86_64 workstation running Ubuntu 22.04 LTS in VMware Workstation. To ensure deterministic compilation, the required host dependencies were installed via `apt` (including `gcc`, `g++`, `diffstat`, `texinfo`, `chrpath`, `zstd`, `cpio`...) and the `en_US.UTF-8` locale was generated with `locale-gen`. Yocto requires the host to run a standard UTF-8 locale so that no errors occur when parsing text.

The target machine is a Raspberry Pi 4 Model B board, which communicates with the host through a USB-UART adapter. The configuration of the components is summarized in Table 3.1.

Building the customized Yocto Linux distribution

The entire process, from preparing the host machine until the board boots, is shown in Figure 3.1. The “No” branch of the decision block is taken when a build fails: the group reads the log, cleans the cache of the failing package with `bitbake -c cleansstate` and builds again.

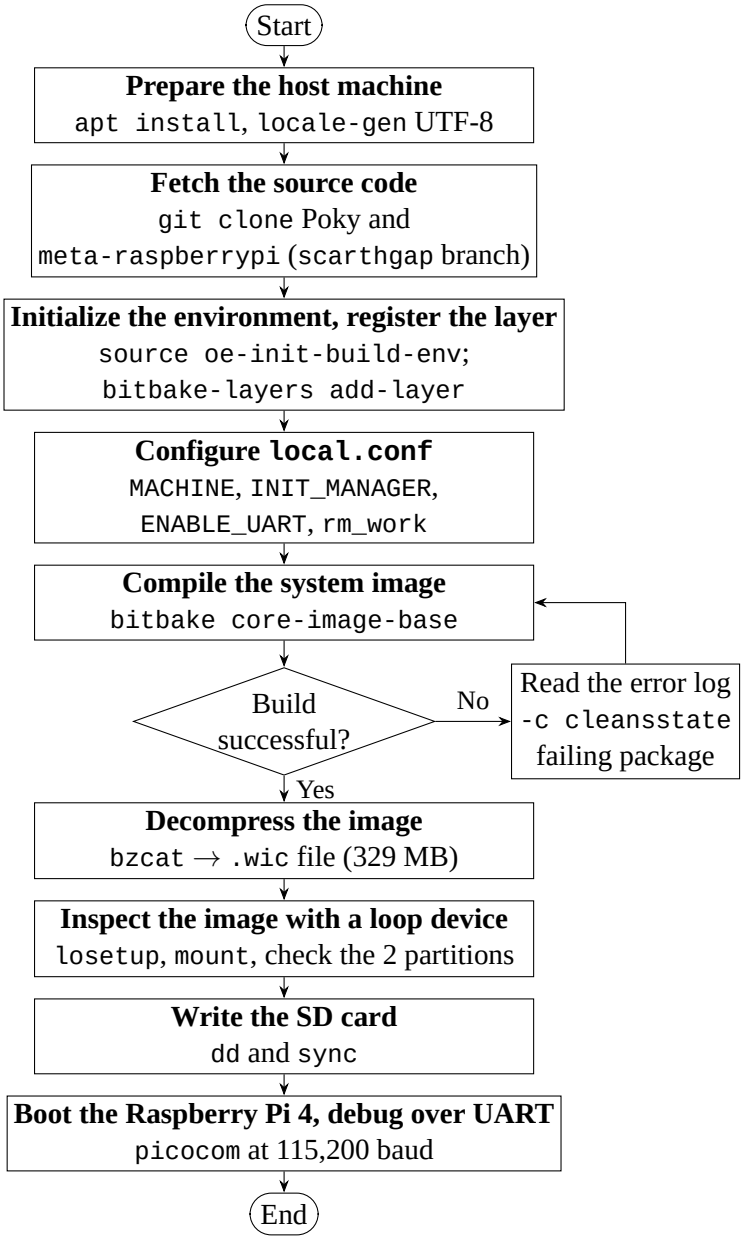


Figure 3.1: Flowchart of building and deploying Yocto Linux for the Raspberry Pi 4

Table 3.1: Experimental environment of the Edge AI part

Component	**Specification**
Host machine	x86_64 workstation, Ubuntu 22.04 LTS running in VMware Workstation
Build system	Yocto Project, Poky Scarthgap 5.0.20 and the `meta-raspberrypi` BSP
Target machine	Raspberry Pi 4 Model B, BCM2711 SoC, 4 GB LPDDR4
System image	`core-image-base`, 329 MB WIC disk image
Linux kernel on the target	6.6.63-v7l (ARMv7l, SMP)
Init system	`systemd`
Debugging	USB-UART adapter (GPIO 14 and 15), `picocom` at 115,200 baud
AI model	MobileNet V1 quantized to `uint8` (TensorFlow Lite 2.16.2)
Dataset	Imagenette, 1,000 validation images, 10 classes

Fetching the source code and initializing the environment

The Poky repository (Scarthgap branch) and the board support package (BSP) layer `meta-raspberrypi` were downloaded into the working directory with `git clone -b scarthgap`. The build environment was initialized with `source oe-init-build-env`; this script sets the `PATH` and `BBPATH` variables for the current shell and then changes into the `build/` directory. The BSP layer was registered in `conf/bblayers.conf` with the `bitbake-layers add-layer` command. The commands executed (the parts in angle brackets are environment-specific parameters) are shown in Listing 3.1.

Listing 3.1: Commands to prepare the environment and register the layer for the Raspberry Pi 4

```
$ sudo apt update
$ sudo apt install -y <package_list>
$ sudo locale-gen en_US.UTF-8
$ git clone -b scarthgap git://git.yoctoproject.org/poky poky-rpi
$ git clone -b scarthgap https://github.com/agherzan/meta-raspberrypi.git
$ cd poky-rpi
$ source oe-init-build-env build
$ bitbake-layers add-layer <path>/meta-raspberrypi
$ bitbake-layers show-layers
```

Build configuration

The global variables are declared in `conf/local.conf` as shown in Listing 3.2.

Listing 3.2: Main settings in `local.conf` for the Raspberry Pi 4

```
1 MACHINE = "raspberrypi4"
2 INIT_MANAGER = "systemd"
3 ENABLE_UART = "1"
4 INHERIT += "rm_work"
```

Compiling the system image

The base Linux image was compiled with the command `bitbake core-image-base`. BitBake fetched the source code and cross-compiled the toolchain and the Linux kernel, and then packaged the image. The process completed successfully with **5,806 tasks**. The `deploy` directory contains the compressed WIC images. The compressed image was decompressed with `bzcat` into a bootable WIC file with a size of **329 MB**.

3.1.2 Image inspection, SD card flashing and UART debugging

So that verification and reproduction can be carried out on a laptop without the physical board, the group set up an emulation procedure using a virtual loop device. The 329 MB WIC file was attached to a loop device with `sudo losetup -Pf`; the `-P` option automatically scans and splits the sub-partitions (`loop10p1`, `loop10p2`). The virtual partitions were then mounted and inspected (Table 3.3).

Table 3.2: Meaning of the configuration variables

Variable	Meaning
`MACHINE = "raspberrypi4"`	Selects the Raspberry Pi 4 as the target machine (`meta-raspberrypi` BSP)
`INIT_MANAGER = "systemd"`	Uses `systemd` as the init system (PID 1)
`ENABLE_UART = "1"`	Enables the hardware UART to provide a serial console for debugging
`INHERIT += "rm_work"`	Deletes the work directory of each recipe after it is built, keeping disk usage at about 28 GB

Table 3.3: Inspection results of the partitions of the WIC image

Item	Partition	Inspection result
Boot partition	`loop10p1`, 130 MB, FAT32	Contains `bcm2711-rpi-4-b.dtb`, `cmdline.txt`, `config.txt`, `start4.elf` and `kernel7l.img`
Root partition	`loop10p2`, 192.4 MB, ext4	Contains the standard POSIX directory tree (`/bin`, `/etc`, `/lib`, `/usr`, `/var`)
PID 1 process	`/sbin/init`	Is a symbolic link pointing to `../lib/systemd/systemd`, showing that `systemd` has been integrated as PID 1

Deployment to the real board and hardware debugging

Flashing the image to the SD card

The block storage devices on the host were listed with `lsblk`, and the partitions of the SD card were unmounted before the low-level write. The WIC image was written raw to the SD card with the `dd` command using a 4 MB block size and the `conv=fsync` option; `sync` then forced the Linux kernel to flush all data still held in the RAM buffers to the card before the card was removed.

UART connection

Serial communication was established with a USB-UART adapter connected to GPIO 14 (TXD) and 15 (RXD) of the Raspberry Pi 4 at **115,200 baud**, providing interactive shell access without an HDMI monitor. The host recognized the adapter as `/dev/ttyUSB0` and used `picocom` to open the session.

Verification on the target machine

Commands run directly on the board confirmed the kernel version, the distribution, the processor architecture and the memory capacity, as shown in Table 3.4.

Table 3.4: Verification results on the Raspberry Pi 4 board

Command	Item	Result
`uname -a`	Linux kernel	6.6.63-v7l, ARMv7l SMP
`cat /etc/os-release`	Distribution	Poky 5.0.20 (Scarthgap)
`cat /proc/cpuinfo`	Processor	Broadcom BCM2711, revision c03115
`free -h`	Memory	About 3.9 GB of LPDDR4 RAM, only 55 MB in use

Build results and characteristics of the system image

Table 3.5 summarizes the results of the build process and the main characteristics of the resulting system image.

Table 3.5: Results of building Yocto Linux for the Raspberry Pi 4

Item	Result
Build system	Poky 5.0.20 “Scarthgap” (LTS), `meta-raspberrypi` BSP
Build command	`bitbake core-image-base`
Tasks executed	5,806 tasks, all successful
Build size	About 28 GB thanks to `rm_work` (could exceed 80 GB without it)
Disk image	329 MB WIC file with two partitions
Partition 1 (boot)	130 MB, FAT32: `bcm2711-rpi-4-b.dtb`, `config.txt`, `cmdline.txt`, `start4.elf`, `kernel7l.img`
Partition 2 (root)	192.4 MB, ext4: standard POSIX directory tree
Init system	`systemd` (PID 1)
Linux kernel	6.6.63-v7l, ARMv7l SMP
Distribution	Poky 5.0.20 (Scarthgap)
RAM when idle	About 55 MB out of about 3.9 GB in total

Reproducibility verification

To verify that the process is reproducible, another member of the group (Tô Quốc Hiệp) independently re-ran the entire build pipeline in parallel (Poky Scarthgap and `meta-raspberrypi`) in a constrained host environment (50 GB disk limit, `rm_work` mandatory). This confirmed that the system image is produced entirely from its description in configuration files and recipes, and does not depend on the particular state of any specific host machine.

3.2 Deploying the Edge AI System on the Raspberry Pi 4

On top of the Yocto Linux operating system built in Section 3.1, this section presents how the system was extended to run deep learning inference directly on the board: adding the TensorFlow Lite layers and the group’s own layer `meta-edge-ai`, loading the MobileNet V1 model, preparing the data and building two inference engines

(Python and native C). This content answers Research Questions 3 and 4.

3.2.1 The `meta-edge-ai` layer and the TensorFlow Lite packages

To support on-device machine learning, the build environment was extended with the `meta-tensorflow-lite` layer, the sub-layers of `meta-openembedded` and the group's own application layer `meta-edge-ai`. The list of layers was verified with `bitbake-layers show-layers`. The combination of layers of the project is shown in Figure 3.2, and the complete software stack on the board in Figure 3.3.

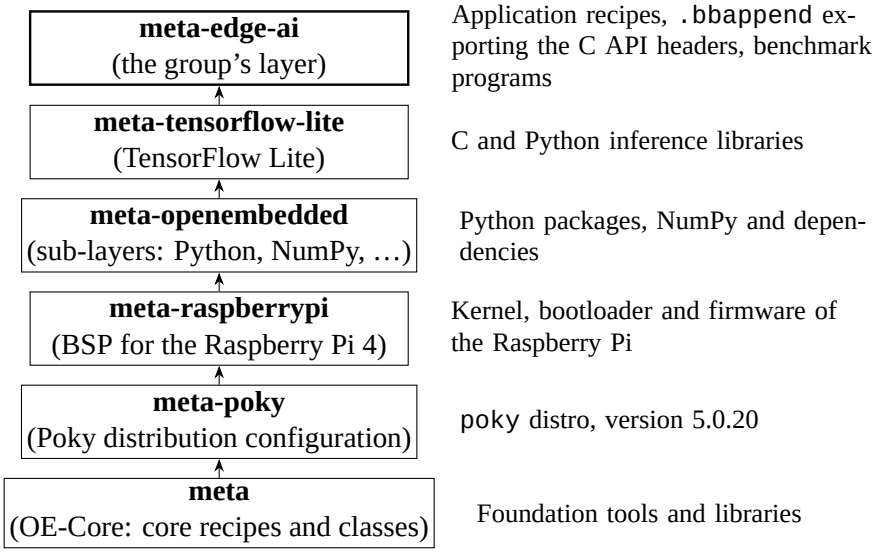


Figure 3.2: Yocto layers of the Raspberry Pi 4 project

The TensorFlow Lite recipes of the upstream layer do not export the development headers of the C API. To fix this, the `meta-edge-ai` layer adds the file `libtensorflow-lite-c_2.16.2.bbappend` (placed in `recipes-framework/`), which installs the headers `c_api.h`, `c_api_types.h` and `builtin_ops.h` into the sysroot of the target machine. The `bitbake-layers show-appends` command was used to verify that the `.bbappend` was correctly linked to the original recipe.

After building the image, the group checked the packaging and deployment (Table 3.6).

3.2.2 The MobileNet V1 model and input data

The MobileNet V1 model

Google's quantized MobileNet V1 model (`model.tflite`, 4.1 MB) and the class dictionary `labels.txt` were loaded onto the target machine, and their integrity was verified with SHA-256 cryptographic hashes. A Python tensor inspection script con-

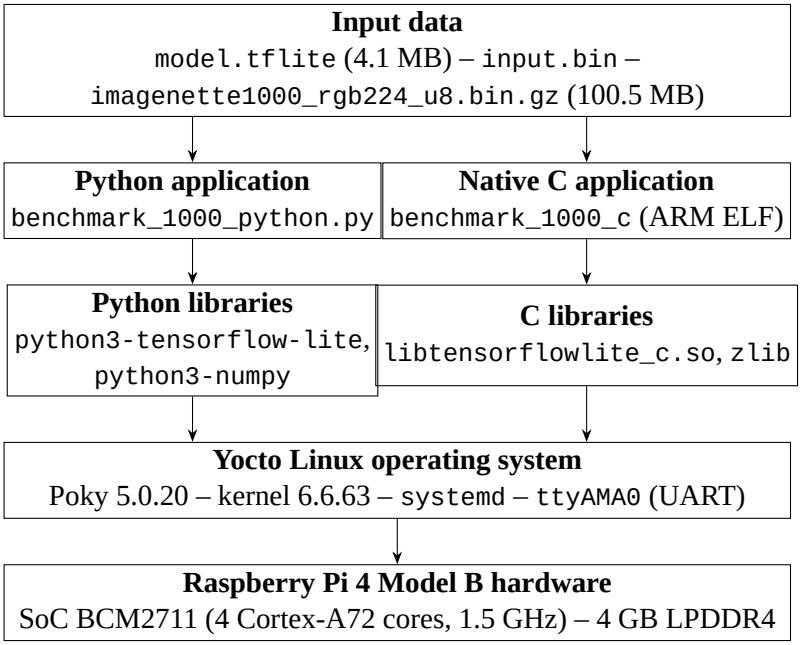


Figure 3.3: Edge AI software stack on the Raspberry Pi 4

Table 3.6: Edge AI packages verified on the target machine

Component	Verification result
`libtensorflow-lite-c`	Present in the package database; shared library `/usr/lib/libtensorflowlite_c.so`, 2.1 MB in size
`python3-tensorflow-lite`	The Python TensorFlow Lite runtime is imported successfully
`python3-numpy`	Version 1.26.4 on the target machine
System resources	Storage space (`df -h /`) and available RAM (`free -h`) are sufficient for the workload

firmed that the model takes an input tensor of shape [1, 224, 224, 3] of type `uint8` and produces an output tensor [1, 1001] of type `uint8`.

Standard test image

The standard Grace Hopper portrait was preprocessed into RGB `uint8` format at a size of 224×224, producing a raw binary stream of $224 \times 224 \times 3 = 150{,}528$ bytes (`input.bin`) and a corresponding NumPy array (`input.npy`). An automated check reported `data_matched: True` between the two files, and `wc -c` confirmed exactly 150,528 bytes.

The 1,000-image Imagenette dataset

For large-scale benchmarking, all 1,000 validation images of the Imagenette dataset were preprocessed and concatenated into a single Gzip-compressed sequential binary file (`imagenette1000_rgb224_u8.bin.gz`, 100.5 MB). The file comes with a metadata manifest `dataset_preparation.json` (image size, SHA-256, class list). In-

tegrity was verified with `gzip -t` and `sha256sum`.

3.2.3 Two inference engines: Python and native C

Two parallel software pipelines were designed to execute the same inference workload.

Python pipeline: written with `tflite_runtime.interpreter`, supporting interactive execution, tensor allocation and output sorting. It consists of `test_real_image.py` (single-image test, Top-5 extraction) and `benchmark_1000_python.py` (1,000-image benchmark).

Native C engine: written in `benchmark_c.c` (218 lines) and `benchmark_1000_c.c` using the TensorFlow Lite C API. The C application bypasses every Python runtime layer, feeds pixel data directly into tensor memory and decompresses the Gzip blocks with `zlib`.

The C programs were packaged with Yocto recipes (e.g., `benchmark-c_1.0.bb` declaring `CFLAGS`, `LDFLAGS` and linking `-ltensorflowlite_c`), and cross-compiled with `bitbake benchmark-c` and `bitbake benchmark-1000-c` without errors. The `file` command confirmed that both are 32-bit ARM ELF executables that run on the Raspberry Pi.

For both engines, each image goes through four sequential stages as shown in Figure 3.4. The time under each block is the actual measured average of the C engine with 1 thread (see Section 4.2); the inference stage is drawn with a bold outline because it accounts for almost all of the time.

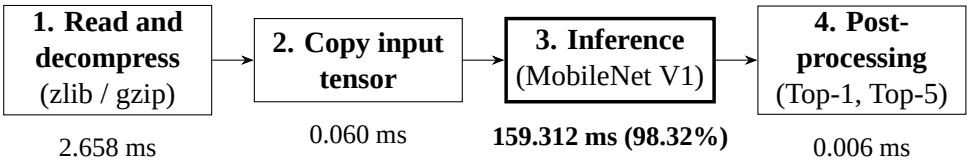


Figure 3.4: The four processing stages of each image and their average times (native C, 1 thread)

3.3 Developing a Device Driver on the QEMU Emulation Platform

Following the instructor's supplementary direction after the model deployment milestone, the group built an additional experimental branch that runs entirely on a QEMU virtual machine: building an embedded Linux image for the `qemux86-64` target with Yocto, writing a character device driver as a kernel module, packaging it with a recipe and integrating it automatically into the system image. This approach makes it possible to develop and test the driver directly on the computer without reflashing the SD card or rebooting the board.

3.3.1 Environment, Yocto image and the `meta-mydriver` layer

The QEMU emulation and device driver development were carried out on a separate personal computer, distinct from the host machine of Sections 3.1 and 3.2, with the configuration shown in Table 3.7. The target machine of this part is the `qemux86-64` virtual machine, not the Raspberry Pi board.

Table 3.7: Configuration of the development machine (host)

Component	Specification
CPU	Intel Core i7-10850H @ 2.70GHz, 12 threads
RAM	32 GB
Host operating system	Ubuntu 26.04.1 LTS (kernel 7.0.0)
Host compiler	GCC 15.2.0, Python 3.14
Yocto Project	Poky 5.0.20 “Scarthgap” (LTS)
Target machine	`qemux86-64`
Target Linux kernel	`linux-yocto` 6.6.151
Emulator	QEMU (version built by Yocto, `qemu-system-native`) + KVM

Setting up the Yocto environment for the QEMU target

Fetching the Poky source code

The Poky source code was downloaded from the official git repository, selecting the `scarthgap` branch:

```
$ mkdir ~/yocto && cd ~/yocto
$ git clone -b scarthgap git://git.yoctoproject.org/poky
$ cd poky
$ source oe-init-build-env build
```

The `oe-init-build-env` script sets up the environment variables (adding `bitbake` and the scripts to `PATH`), creates the `build/` directory together with sample configuration files and changes into that directory. The script must be run again every time a new terminal is opened.

Organization of the project directory

The project directory is organized as shown in Figure 3.5. The source download directory (`downloads`) and the cache (`sstate-cache`) are placed outside `poky/` so that they can be reused when the build directory is deleted or Poky is upgraded.

Build configuration (`local.conf`)

The file `build/conf/local.conf` contains the settings for the build process. The main settings used in this project are presented in Listing 3.3.

```
~/yocto/
|-- poky/                       # Yocto source code (scarthgap branch)
|   |-- bitbake/                # Build tool
|   |-- meta/  meta-poky/  meta-yocto-bsp/
|   |-- oe-init-build-env
|   `-- build/                  # Build directory
|       |-- conf/local.conf
|       |-- conf/bblayers.conf
|       `-- tmp/deploy/images/qemux86-64/   # Output
|-- meta-mydriver/              # Custom layer containing the driver
|-- downloads/                  # DL_DIR (~5.6 GB)
`-- sstate-cache/               # SSTATE_DIR (~4.2 GB)
```

Figure 3.5: Structure of the project directory

Listing 3.3: Main settings in `local.conf`

```
MACHINE ?= "qemux86-64"
DISTRO ?= "poky"
DL_DIR = "${HOME}/yocto/downloads"
SSTATE_DIR = "${HOME}/yocto/sstate-cache"
BB_NUMBER_THREADS = "8"
PARALLEL_MAKE = "-j 8"
INHERIT += "rm_work"
EXTRA_IMAGE_FEATURES ?= "debug-tweaks"
PACKAGECONFIG:append:pn-qemu-system-native = " sdl"

# --- meta-mydriver ---
IMAGE_INSTALL:append = " mychardev"
EXTRA_IMAGE_FEATURES += "ssh-server-dropbear"
```

The meaning of these settings is explained in Table 3.8.

Table 3.8: Meaning of the configuration variables in `local.conf`

Variable	Meaning
`MACHINE`	Target machine: 64-bit x86 virtual machine for QEMU
`DISTRO`	Uses the Poky reference distribution policy
`DL_DIR`	Where the downloaded source code is stored
`SSTATE_DIR`	Where the task result cache is stored
`BB_NUMBER_THREADS`	Number of BitBake tasks run in parallel
`PARALLEL_MAKE`	Number of threads for each `make` command
`INHERIT += "rm_work"`	Deletes the work directory of each recipe after building to save disk space
`debug-tweaks`	Allows the `root` account to log in without a password (for development only)
`IMAGE_INSTALL:append`	Adds the `mychardev` package to the system image
`ssh-server-dropbear`	Adds the lightweight Dropbear SSH server for accessing the virtual machine

Syntax note: the operator `?=` assigns only when the variable has no value yet; `+=` appends with an inserted space; the suffix `:append` is applied last and does **not** insert a space automatically, so the value must start with a space (`" mychardev"`).

Building the base system image

The minimal system image is built with the command:

```
$ bitbake core-image-minimal
```

On the first build, BitBake downloads all the source code (about 5.6 GB) and compiles from scratch the cross toolchain, the Linux kernel, the C library, BusyBox, the init system… This process takes several hours depending on the machine configuration. Subsequent builds are much faster thanks to sstate-cache.

Designing the `meta-mydriver` layer

Creating and registering the layer

Following the principle of not modifying existing layers, the driver is placed in its own layer. The layer is created and registered with the `bitbake-layers` tool:

```
$ bitbake-layers create-layer ~/yocto/meta-mydriver
$ bitbake-layers add-layer ~/yocto/meta-mydriver
```

The `add-layer` command adds the layer path to the `BBLAYERS` variable in `bblayers.conf` (Listing 3.4).

Listing 3.4: Layer list in `bblayers.conf`

```
BBLAYERS ?= " \
  /home/ngan/yocto/poky/meta \
  /home/ngan/yocto/poky/meta-poky \
  /home/ngan/yocto/poky/meta-yocto-bsp \
  /home/ngan/yocto/meta-mydriver \
  "
```

Layer structure

The structure of the completed layer is presented in Figure 3.6. The recipe is placed in the `recipes-kernel/` group according to Yocto convention; the source files are placed in the `files/` subdirectory so that BitBake finds them automatically through the `file://` prefix.

```
meta-mydriver/
|-- conf/
|   `-- layer.conf              # Layer declaration
|-- COPYING.MIT
|-- README
`-- recipes-kernel/
    `-- mychardev/
        |-- mychardev_0.1.bb    # Recipe (PN=mychardev, PV=0.1)
        `-- files/
            |-- mychardev.c     # Driver source code
            |-- Makefile        # Kbuild Makefile
            `-- COPYING         # GPL-2.0 license
```

Figure 3.6: Structure of the `meta-mydriver` layer

The file `conf/layer.conf` (Listing 3.5) declares the recipe search pattern `recipes-*/*/*.bb`, the layer priority and the compatible Yocto version

`scarthgap`.

Listing 3.5: The file `meta-mydriver/conf/layer.conf`

```
BBPATH .= ":${LAYERDIR}"
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"
BBFILE_COLLECTIONS += "meta-mydriver"
BBFILE_PATTERN_meta-mydriver = "^${LAYERDIR}/"
BBFILE_PRIORITY_meta-mydriver = "6"
LAYERDEPENDS_meta-mydriver = "core"
LAYERSERIES_COMPAT_meta-mydriver = "scarthgap"
```

3.3.2 Design and implementation of the `mychardev` driver

Functional specification

The `mychardev` driver is designed with the following requirements:

When loaded, create the device file `/dev/mychardev` with read/write permission for all users (`0666`).

Data written to the device is stored in a 1024-byte buffer in kernel memory; a read operation returns the written data.

Fully support the `open`, `read`, `write`, `release` and `llseek` operations.

Protect the buffer with a mutex when several processes access it concurrently.

Log every operation to the kernel message buffer (viewable with `dmesg`).

Cleanly release all resources when the module is removed (`rmmod`).

Operation diagram

The lifecycle flowchart of the driver is shown in Figure 3.7. When loaded, the driver registers a misc device; if registration fails, the init function returns an error code and the module is not loaded. If it succeeds, the driver serves calls from applications until a module removal command is issued.

Source code implementation

The full source code of the driver is presented in Appendix B. This section analyzes its main components.

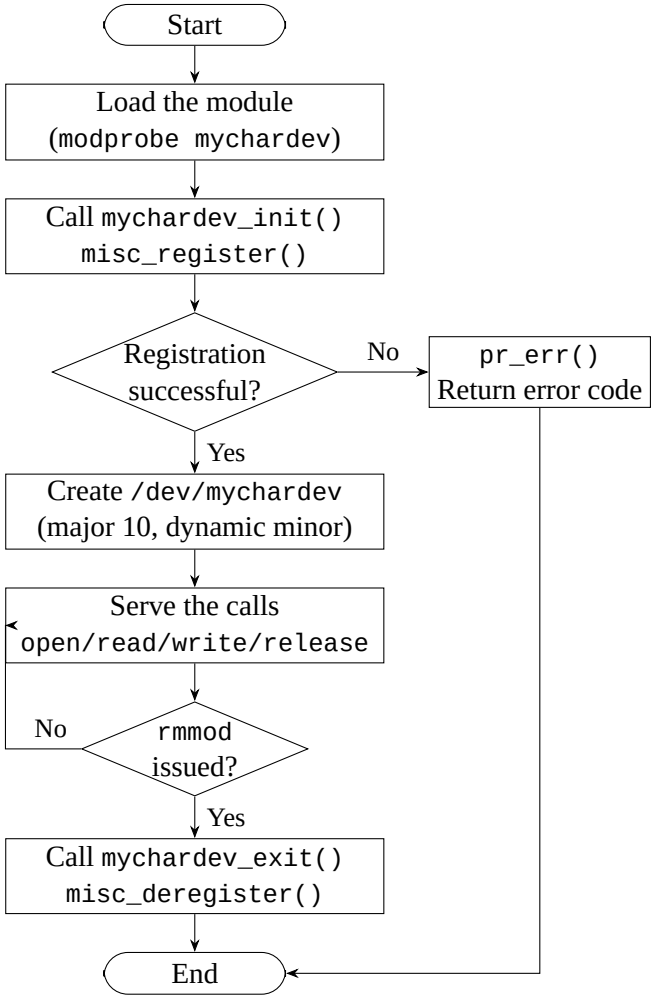


Figure 3.7: Lifecycle flowchart of the `mychardev` driver

Data declarations

Listing 3.6: Declaration of the buffer, the length and the mutex

```
#define DEVICE_NAME "mychardev"
#define BUF_SIZE    1024

static char dev_buf[BUF_SIZE];
static size_t dev_len;
static DEFINE_MUTEX(dev_lock);
```

The variable `dev_buf` is the data buffer, `dev_len` holds the number of valid bytes currently in the buffer, and `dev_lock` is the lock protecting these two variables. The `static` keyword limits the scope of these symbols to the file, avoiding name collisions with the rest of the kernel.

Read function

Listing 3.7: Read operation handler

```
static ssize_t mychardev_read(struct file *file, char __user *ubuf,
                              size_t count, loff_t *ppos)
{
        ssize_t ret;

        mutex_lock(&dev_lock);
        ret = simple_read_from_buffer(ubuf, count, ppos, dev_buf, dev_len);
        mutex_unlock(&dev_lock);
        return ret;
}
```

The parameter `ubuf` points to the memory of the user process, `count` is the number of bytes requested, and `ppos` is the current position in the file. The function `simple_read_from_buffer()` copies data from `dev_buf` starting at `*ppos` to user space (internally using `copy_to_user()`), updates `*ppos` and returns the number of bytes copied. When `*ppos` reaches `dev_len`, the function returns 0 – signaling the end of file (EOF), which lets programs such as `cat` know when to stop reading.

Write function

Listing 3.8: Write operation handler

```
static ssize_t mychardev_write(struct file *file, const char __user *ubuf,
                               size_t count, loff_t *ppos)
{
        ssize_t ret;

        mutex_lock(&dev_lock);
        ret = simple_write_to_buffer(dev_buf, BUF_SIZE, ppos, ubuf, count);
        if (ret > 0 && *ppos > dev_len)
                dev_len = *ppos;
        mutex_unlock(&dev_lock);
        if (ret > 0)
                pr_info(DEVICE_NAME ": wrote %zd bytes\n", ret);
        return ret;
}
```

The function `simple_write_to_buffer()` copies data from the user into the buffer (internally using `copy_from_user()`), ensuring that `BUF_SIZE` is not exceeded. After the write, the valid data length `dev_len` is updated if the write position goes beyond the old length.

The flowcharts of the read and write functions are presented in Figure 3.8 and Figure 3.9 respectively. Both functions lock the mutex before accessing the buffer and unlock it before returning the result, ensuring that no two processes operate on `dev_buf` at the same time.

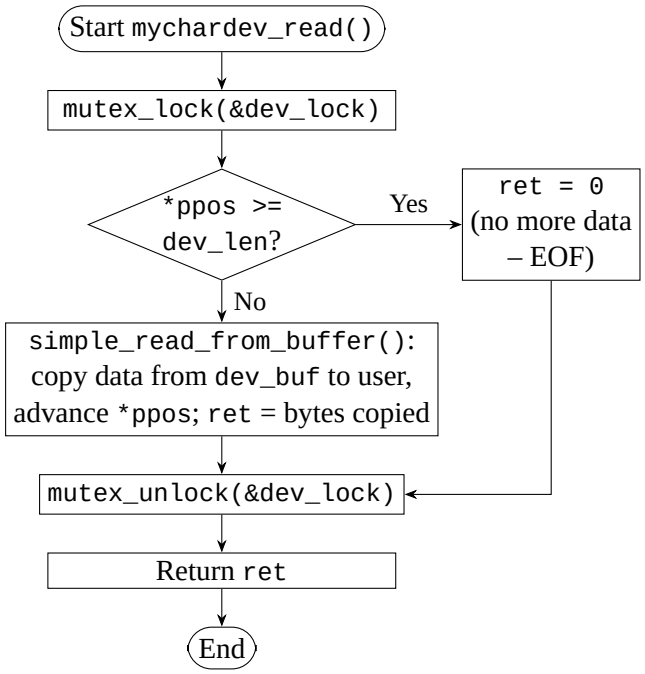


Figure 3.8: Flowchart of the `mychardev_read()` function

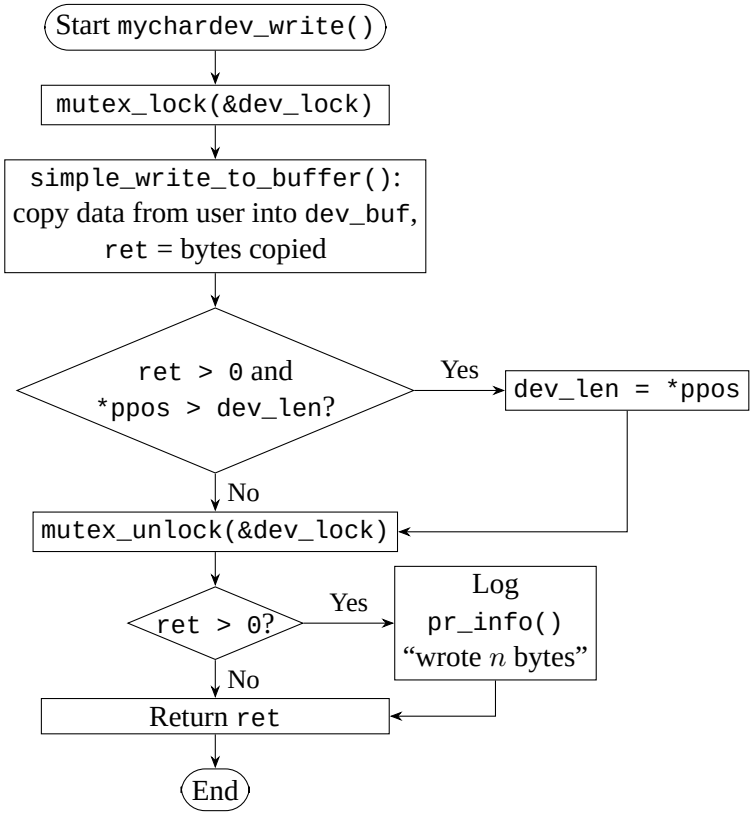


Figure 3.9: Flowchart of the `mychardev_write()` function

File operations table and device registration

Listing 3.9: The `file_operations` and `miscdevice` structures

```
static const struct file_operations mychardev_fops = {
        .owner   = THIS_MODULE,
        .open    = mychardev_open,
        .release = mychardev_release,
        .read    = mychardev_read,
        .write   = mychardev_write,
        .llseek  = default_llseek,
};

static struct miscdevice mychardev_misc = {
        .minor = MISC_DYNAMIC_MINOR,
        .name  = DEVICE_NAME,
        .fops  = &mychardev_fops,
        .mode  = 0666,
};
```

The `file_operations` structure maps system calls to the corresponding functions of the driver. The field `.owner = THIS_MODULE` lets the kernel increment the module's reference count while the device is open, preventing the module from being removed while it is in use. The `miscdevice` structure declares the device name (which is also the file name in `/dev`), requests a dynamically allocated minor number and sets the access permissions to `0666`.

Init and exit functions

Listing 3.10: Module init and exit functions

```
static int __init mychardev_init(void)
{
        int ret = misc_register(&mychardev_misc);

        if (ret) {
                pr_err(DEVICE_NAME ": misc_register failed: %d\n", ret);
                return ret;
        }
        pr_info(DEVICE_NAME ": loaded, minor=%d\n", mychardev_misc.minor);
        return 0;
}

static void __exit mychardev_exit(void)
{
        misc_deregister(&mychardev_misc);
        pr_info(DEVICE_NAME ": unloaded\n");
}

module_init(mychardev_init);
module_exit(mychardev_exit);
MODULE_LICENSE("GPL");
```

The `__init` macro allows the kernel to free the code of the init function once loading is complete. An important principle is that every resource registered in the init function must be released in the exit function; otherwise, after the module is removed, the kernel still holds pointers into code that has been freed, which can cause serious errors. The declaration `MODULE_LICENSE("GPL")` is mandatory for the module to be allowed to use kernel symbols that are exported only to GPL code.

Kbuild-standard Makefile

A kernel module is not compiled directly with `gcc` but through the kernel build system (Kbuild). The driver's Makefile is presented in Listing 3.11.

Listing 3.11: Makefile of the driver

```
obj-m := mychardev.o

SRC := $(shell pwd)

all:
	$(MAKE) -C $(KERNEL_SRC) M=$(SRC)

modules_install:
	$(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules_install
```

`obj-m := mychardev.o`: tells Kbuild to compile `mychardev.c` into a module (`m`); with `obj-y` the code would be linked permanently into the kernel.

`-C $(KERNEL_SRC)`: changes into the configured kernel source directory; this variable is passed in by Yocto and points to `STAGING_KERNEL_DIR`.

`M=$(SRC)`: tells Kbuild where the out-of-tree module is located.

`modules_install`: installs the `.ko` file into `$(INSTALL_MOD_PATH)/lib/modules/<kernel version>/updates/` – the default directory for out-of-tree modules.

The Kbuild compilation proceeds as follows: `mychardev.c` → `mychardev.o` → (`modpost`) `mychardev.mod.c` → `mychardev.mod.o` → `mychardev.ko`.

3.3.3 *Recipe, image integration and running on QEMU*

The entire transformation chain from source code to the `/dev/mychardev` device in the virtual machine is summarized in Figure 3.10; to the right of each step is the corresponding BitBake task.

Recipe contents

The recipe `mychardev_0.1.bb` (Listing 3.12) describes how BitBake fetches the source code, compiles and packages the driver.

Listing 3.12: Recipe `mychardev_0.1.bb`

```
SUMMARY = "Simple character device driver (/dev/mychardev)"
DESCRIPTION = "Example out-of-tree Linux kernel module built with Yocto"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://COPYING;md5=12f884d2ae1ff87c09e5b7ccc2c4ca7e"

inherit module

SRC_URI = "file://Makefile \
           file://mychardev.c \
           file://COPYING \
          "

S = "${WORKDIR}"

```

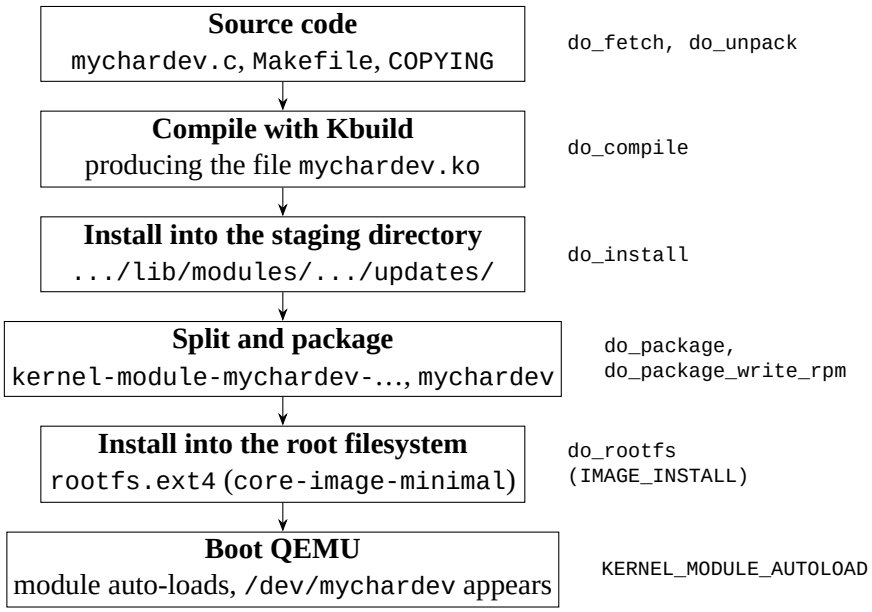


Figure 3.10: Path of the driver from source code to the QEMU virtual machine

```
15 RPROVIDES:${PN} += "kernel-module-mychardev"
16
17 # Load the module automatically at boot
18 KERNEL_MODULE_AUTOLOAD += "mychardev"
```

The meaning of each component is presented in Table 3.9.

Table 3.9: Explanation of the components of the recipe `mychardev_0.1.bb`

Component	Meaning
`LICENSE`, `LIC_FILES_CHKSUM`	Declares the license and the MD5 checksum of the license file; if the file is modified, the build reports an error
`inherit module`	Inherits the class `module.bbclass`: automatically adds a dependency on the kernel, defines `do_compile`, `do_install` and splits off the module package
`SRC_URI`	List of source files; `file://` takes them from the `files/` directory next to the recipe
`S = "${WORKDIR}"`	The source directory after unpacking is the work directory itself
`RPROVIDES`	The `mychardev` package additionally provides the name `kernel-module-mychardev` so that other recipes can declare a dependency on it
`KERNEL_MODULE_AUTOLOAD`	Creates the file `/etc/modules-load.d/mychardev.conf` so that the module is loaded automatically at boot

Role of the `module` class

The class `module.bbclass` in OE-Core takes care of most of the work:

1. Adds `DEPENDS += "virtual/kernel"` so that the kernel is built before the module.
2. In `do_compile`: runs `make` with `KERNEL_SRC` pointing to the built kernel

source, together with the cross-compilation variables (`CC`, `LD`, `ARCH`…).

3. In `do_install`: runs `make modules_install` with `INSTALL_MOD_PATH=${D}`.
4. In `do_package`: through the `kernel-module-split` class, splits each `.ko` file into its own package named `kernel-module-mychardev-6.6.151-yocto-standard`; the main package `mychardev` becomes a "meta" package that pulls in the module package.
5. After the package is installed: runs `depmod` so that `modprobe` can find the module.

Integration into the system image and build

The driver is added to the system image through the line `IMAGE_INSTALL:append = " mychardev"` in `local.conf`, after which the image is rebuilt:

```
$ bitbake core-image-minimal
```

BitBake automatically detects the new recipe, builds the driver and regenerates the root filesystem. The tasks executed for the `mychardev` recipe are summarized in Table 3.10.

Table 3.10: BitBake tasks executed for the `mychardev` recipe

Task	Result
`do_fetch`, `do_unpack`	Copies the 3 source files into `tmp/work/qemux86_64-poky-linux/mychardev/0.1/`
`do_compile`	Compiles `mychardev.ko`
`do_install`	`image/lib/modules/6.6.151-yocto-standard/updates/mychardev.ko` and `image/etc/modules-load.d/mychardev.conf`
`do_package`	Splits the packages: `kernel-module-mychardev-…`, `mychardev`, `-dbg`, `-dev`, `-src`
`do_package_write_rpm`	Creates the `.rpm` files in `tmp/deploy/rpm/qemux86_64/`
`core-image-minimal: do_rootfs`	Installs the driver package into the root filesystem

Running the system on QEMU

The system image is booted on QEMU with the `runqemu` script:

```
$ runqemu qemux86-64 nographic slirp kvm
```

The script automatically generates the corresponding QEMU command: a `q35` virtual machine with 4 CPU cores, 256 MB of RAM, a virtio disk containing `rootfs.ext4`, the `bzImage` kernel loaded directly, and user-mode networking forwarding port `127.0.0.1:2222` → `22`. The architecture of the emulation environment is shown in

Figure 3.11 and the parameters of the virtual machine in Table 3.11. After booting, the user can log in via the serial console or via SSH with the passwordless `root` account:

```
$ ssh -p 2222 root@127.0.0.1
```

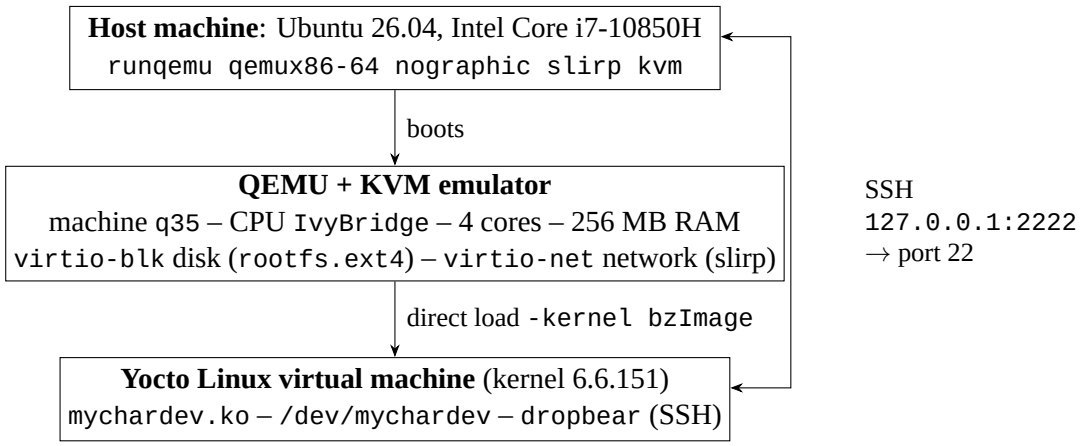


Figure 3.11: Architecture of the QEMU emulation environment

Table 3.11: Parameters of the QEMU virtual machine generated by `runqemu`

Component	**Value**
Yocto `MACHINE`	`qemux86-64` (generic x86 virtual machine, not a real board)
Architecture	x86-64, run with `qemu-system-x86_64`
QEMU machine type	`q35` (Intel Q35 chipset, with PCIe)
Emulated CPU	`IvyBridge`, 4 cores (`-smp 4`)
RAM	256 MB (`-m 256`)
Acceleration	KVM (`-enable-kvm`)
Disk	`virtio-blk` containing `rootfs.ext4`
Network	`virtio-net-pci`, user-mode, port forwarding 2222 → 22
Kernel loading	Directly via `-kernel bzImage`, without a bootloader
Console	Serial port `ttyS0`

CHAPTER 4

EXPERIMENTAL RESULTS AND EVALUATION

4.1 Edge AI Inference Performance on the Raspberry Pi 4

This chapter presents the measurement results on the Raspberry Pi 4 board (Sections 4.1 and 4.2) and the QEMU emulation and driver testing results (Section 4.3). All timing figures are actual averages measured on the board.

Single-image baseline: Python and native C

Running both single-image benchmarks on the board confirmed that the two engines give the same result on the Grace Hopper image: both predict the class `military uniform` (class 653) with a score of 227, corresponding to a confidence of 89.02%. The measurement results are summarized in Table 4.1.

Table 4.1: Comparison of Python and native C for single-image inference with 1 thread

Metric	Python	Native C	Assessment
Mean inference latency	161.113 ms	160.320 ms	C is 0.793 ms faster (0.49% lower); the two engines are algorithmically equivalent
Median latency	160.397 ms	160.297 ms	Difference of 0.100 ms (0.06% lower)
95th percentile latency (P95)	162.791 ms	160.539 ms	C has a tighter tail bound (2.252 ms lower)
Peak RAM (RSS)	31.37 MiB	10.12 MiB	C saves 21.25 MiB (67.7% less memory)
Classification result	Class 653 (score 227)	Class 653 (score 227)	Exact mathematical match

The results show that the inference latencies of the two engines almost coincide, while their memory usage differs markedly. This is consistent with the theoretical analysis: the heavy computation resides in the TensorFlow Lite library (written in C++) shared by both, whereas the overhead of Python lies mainly in the memory of the interpreter.

Multi-core scaling and thread parallelism

To evaluate hardware-level optimization without changing the model weights, both runtimes were measured with 1, 2 and 4 CPU threads (the `num_threads` parameter). The results are in Table 4.2 and Figure 4.1.

Going from 1 to 4 threads reduces latency by about 72.5% for Python (from 161.030

Table 4.2: Mean latency by number of CPU threads

Threads	Python	Native C	Speedup (C)	CPU usage
1 thread (1 core)	161.030 ms	159.138 ms	1.00× (baseline)	99.93% (1 core)
2 threads (2 cores)	83.459 ms	82.799 ms	1.92×	199.89% (2 cores)
4 threads (4 cores)	44.219 ms	43.326 ms	3.67×	398.98% (4 cores)

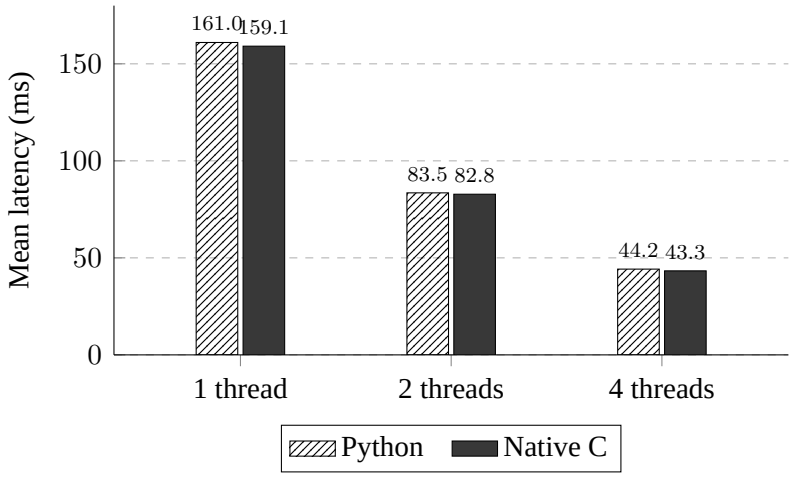


Figure 4.1: Mean inference latency by number of threads (lower is better)

ms to 44.219 ms) and by about 72.8% for native C (from 159.138 ms to 43.326 ms), corresponding to a speedup of about 3.65 to 3.67 times, very close to the ideal limit of 4 times for 4 cores. CPU usage grows almost linearly with the number of threads (from 99.93% to 398.98%), showing that the cores are utilized almost completely.

Large-scale benchmark with 1,000 images

All 1,000 Imagenette validation images were evaluated on the board with the 1-thread and 4-thread configurations for both engines. The results are in Table 4.3 and Figures 4.2, 4.3 and 4.4.

Table 4.3: Results of the 1,000-image benchmark

Metric	Python (1 thread)	Native C (1 thread)	Python (4 threads)	Native C (4 threads)
Total time (1,000 images)	164.399 s	162.040 s	47.184 s	46.402 s
Mean inference latency	161.011 ms	159.312 ms	43.957 ms	43.672 ms
Throughput	6.083 images/s	6.171 images/s	21.193 images/s	21.551 images/s
CPU usage	99.89%	99.93%	390.20%	394.07%
Peak memory (RSS)	33.72 MiB	10.25 MiB	33.72 MiB	10.25 MiB

With 4 threads, the total processing time drops from about 164 seconds to about 46 to 47 seconds (a reduction of about 71%). The native C engine alone reaches 21.55

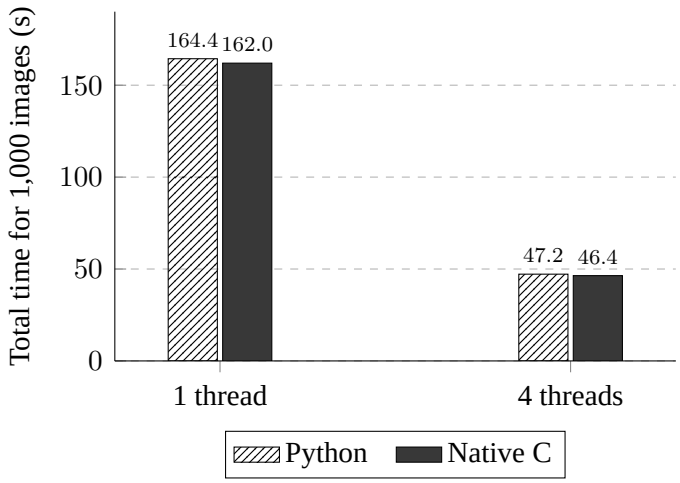


Figure 4.2: Total time to process 1,000 images (lower is better)

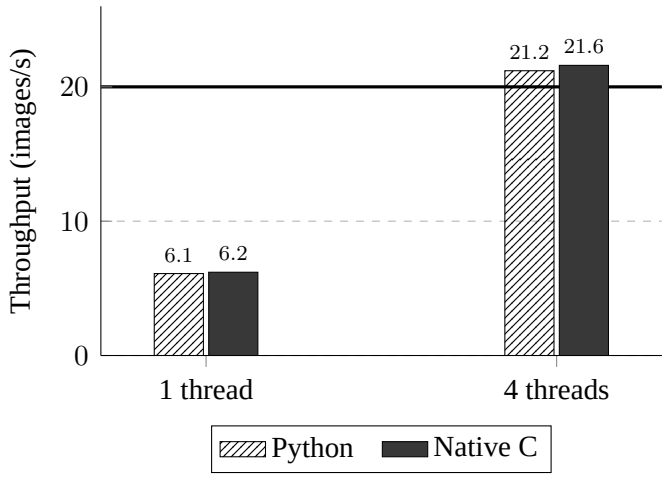


Figure 4.3: Throughput when processing 1,000 images (the bold horizontal line is the real-time threshold of 20 images/s)

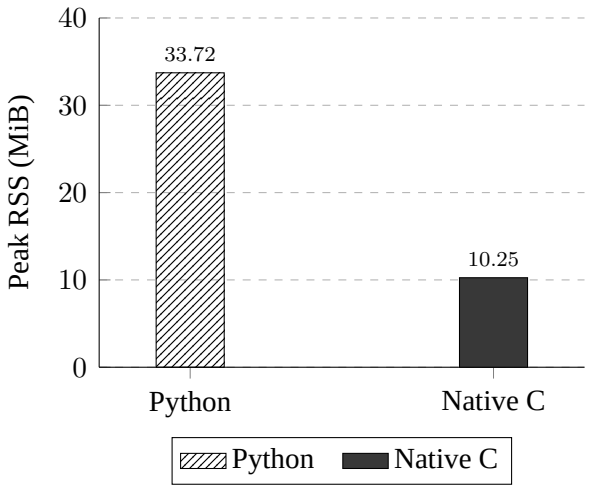


Figure 4.4: Peak resident set size (RSS): native C needs only 30.4% of Python's memory

images/s, exceeding the real-time threshold of 20 images/s set in Chapter 1, while Python reaches 21.19 images/s. In terms of memory, native C uses only 10.25 MiB, i.e., 30.4% of the 33.72 MiB used by Python.

An automated line-by-line check across all 1,000 prediction records confirmed an exact match with **0 discrepancies**: thread parallelism does not change the classification results.

4.2 Bottlenecks, CPU Governor and Accuracy

To answer the research question on the system bottleneck, the per-image latency was measured separately for the four stages (Figure 3.4). The results are in Table 4.4.

Table 4.4: Breakdown of the per-image processing time by stage

Stage	Python (1 thread)	Native C (1 thread)	Python (4 threads)	Native C (4 threads)
Read / decompress	2.962 ms	2.658 ms	2.835 ms	2.657 ms
Copy input tensor	0.255 ms	0.060 ms	0.223 ms	0.063 ms
Inference (MobileNet)	161.011 ms	159.312 ms	43.957 ms	43.672 ms
Post-processing	0.101 ms	0.006 ms	0.102 ms	0.006 ms
Total latency per image	164.329 ms	162.036 ms	47.117 ms	46.398 ms
Share of inference	97.98%	98.32%	93.30%	94.12%

Bottleneck conclusion. The data show that the inference step accounts for **93.30% to 98.32%** of the total processing time of each image. Data decompression, tensor copying and post-processing together account for less than 6.7% of the latency. Therefore, the arithmetic computation of the CPU is the main bottleneck, and multi-thread scaling across several cores is the most effective optimization direction.

Operating system optimization: the CPU frequency governor

The group investigated system-level frequency tuning by comparing the default `ondemand` governor (frequency varying from 600 MHz to 1.5 GHz) with `performance` (forcing all four Cortex-A72 cores to run at 1.5 GHz at all times). After the experiment, the `ondemand` governor was restored. The results are in Table 4.5.

The `performance` mode brings no significant improvement (under 1%) but raises the temperature by an additional 8.8 to 19.0 °C. Therefore, for this workload, keeping the `ondemand` governor is the more reasonable choice in terms of heat and power.

Model accuracy and global verification

The Top-1 classification accuracy of the model was evaluated on the 1,000 Imagenette images, reaching **69.10%** (691 correct predictions out of 1,000). The per-class results are in Table 4.6.

Table 4.5: Comparison of the `ondemand` and `performance` governors

Configuration and metric	**ondemand**	**performance**	**Impact**
Python 4 threads: mean latency	44.011 ms	43.860 ms	-0.151 ms (0.34% improvement)
Python 4 threads: total time	47.402 s	47.012 s	-0.390 s (0.82% improvement)
Native C 4 threads: mean latency	43.581 ms	43.607 ms	$+0.026$ ms (statistical noise)
Native C 4 threads: total time	46.316 s	46.325 s	$+0.009$ s (negligible difference)
Operating temperature range	46.2 °C → 61.8 °C	65.2 °C → 70.6 °C	8.8 °C to 19.0 °C hotter

Table 4.6: Classification accuracy for each of the 10 Imagenette classes

Class (image name)	**ImageNet index**	**Correct predictions (/100)**	**Accuracy**
Tench (fish)	1	91 / 100	91.00%
English Springer (dog)	218	54 / 100	54.00%
Cassette Player	483	54 / 100	54.00%
Chain Saw	492	65 / 100	65.00%
Church	498	32 / 100	32.00%
French Horn	567	72 / 100	72.00%
Garbage Truck	570	70 / 100	70.00%
Gas Pump	572	78 / 100	78.00%
Golf Ball	575	91 / 100	91.00%
Parachute	702	84 / 100	84.00%
Total (10 classes)		**691 / 1,000**	**69.10%**

The global verification confirmed that all 8 experimental configurations produced identical predictions, **with no discrepancies at all** (all confirmed), and the experimental parameters together with the cryptographic hashes were stored in the file `experiment_record.txt`. Table 4.7 summarizes all metrics of the four main configurations.

4.3 QEMU Emulation Results and Driver Testing

Build results for the QEMU target machine

After adding the `meta-mydriver` layer and the SSH server, the `bitbake core-image-minimal` command completed successfully. The BitBake summary is presented in Listing 4.1.

Table 4.7: Summary of the Edge AI metrics

Metric	Python 1 thread	Python 4 threads	Native C 1 thread	Native C 4 threads
Total time (1,000 images)	164.399 s	47.184 s	162.040 s	46.402 s
Mean inference latency	161.011 ms	43.957 ms	159.312 ms	43.672 ms
Throughput	6.083 fps	21.193 fps	6.171 fps	21.551 fps
CPU usage	99.89%	390.20%	99.93%	394.07%
Peak memory (RSS)	33.72 MiB	33.72 MiB	10.25 MiB	10.25 MiB
Top-1 accuracy	69.10%	69.10%	69.10%	69.10%
Prediction consistency	Reference baseline	100% match (0 discrepancies)	100% match (0 discrepancies)	100% match (0 discrepancies)

Listing 4.1: BitBake summary output

```
WARNING: Host distribution "ubuntu-26.04" has not been validated with
this version of the build system; you may possibly experience
unexpected failures.
NOTE: Tasks Summary: Attempted 4657 tasks of which 4552 didn't need
to be rerun and all succeeded.
Summary: There was 1 WARNING message.
```

Of the 4657 tasks in total, 4552 tasks (about 97.7%) were retrieved from sstate-cache; only 105 tasks related to the driver, Dropbear and the regeneration of the root filesystem actually had to be executed. This demonstrates the effectiveness of BitBake's signature-based caching mechanism. The single warning indicates that Ubuntu 26.04 is not yet on the list of officially validated host operating systems for Scarthgap; this warning does not affect the build results.

Output files

The output files are located in the directory `tmp/deploy/images/qemux86-64/` (Table 4.8).

Table 4.8: Main output files of the build process

File	Size	Description
`bzImage`	12 MB	Linux kernel 6.6.151
`core-image-minimal-qemux86-64.rootfs.ext4`	40 MB	Root filesystem (disk image)
`core-image-minimal-….rootfs.tar.bz2`	18 MB	Compressed root filesystem
`core-image-minimal-….rootfs.manifest`	–	List of the 41 packages in the image
`core-image-minimal-….qemuboot.conf`	–	Boot configuration for `runqemu`
`modules-qemux86-64.tgz`	–	Kernel modules

Inspecting the manifest file confirms that the driver and the SSH server have been integrated into the system image:

```
$ grep -E 'mychardev|dropbear' core-image-minimal-qemux86-64.rootfs.manifest
dropbear core2_64 2022.83
kernel-module-mychardev-6.6.151-yocto-standard qemux86_64 0.1
mychardev qemux86_64 0.1
packagegroup-core-ssh-dropbear noarch 1.0
```

The software packages of the driver generated by BitBake:

```
$ ls tmp/deploy/rpm/qemux86_64/ | grep mychardev
kernel-module-mychardev-6.6.151-yocto-standard-0.1-r0.qemux86_64.rpm
mychardev-0.1-r0.qemux86_64.rpm
mychardev-dbg-0.1-r0.qemux86_64.rpm
mychardev-dev-0.1-r0.qemux86_64.rpm
mychardev-src-0.1-r0.qemux86_64.rpm
```

Booting the system on QEMU

The system booted successfully on QEMU with KVM acceleration. The Linux kernel completed its initialization after about 1.3 seconds, and the init process (SysVinit 3.04) started the `udev`, `syslogd` and `dropbear` services and displayed the login prompt (Listing 4.2).

Listing 4.2: Boot messages on the serial console (excerpt)

```
[    0.013093] Kernel command line: root=/dev/vda rw ip=dhcp console=ttyS0 ...
[    1.294012] Freeing unused kernel image (initmem) memory: 3448K
INIT: version 3.04 booting
Starting udev
[    1.342471] udevd[127]: starting version 3.2.14
[    1.718285] mychardev: loading out-of-tree module taints kernel.
[    1.722027] mychardev: loaded, minor=259
...
Starting syslogd/klogd: done

Poky (Yocto Project Reference Distro) 5.0.20 qemux86-64 /dev/ttyS0

qemux86-64 login: root
root@qemux86-64:~# uname -a
Linux qemux86-64 6.6.151-yocto-standard #1 SMP PREEMPT_DYNAMIC x86_64 GNU/Linux
```

It can be seen that the `mychardev` driver was loaded automatically at second 1.72 – during the boot phase – thanks to the `KERNEL_MODULE_AUTOLOAD` setting in the recipe. The message “*loading out-of-tree module taints kernel*” is not an error but a kernel note indicating that a module outside the official source tree has been loaded.

Testing the `mychardev` driver

Test scenarios

The test cases were designed to check the entire lifecycle and functionality of the driver (Table 4.9).

Table 4.9: Test cases for the `mychardev` driver

No.	Test command	Expected result	Result
1	`lsmod`	The `mychardev` module has been loaded	Pass
2	`modinfo mychardev`	Correct version, description, `updates/` path	Pass
3	`ls -l /dev/mychardev`	Character device, major 10, permissions `rw-rw-rw-`	Pass
4	`echo ... > /dev/mychardev`	Write succeeds, `dmesg` records the number of bytes	Pass
5	`cat /dev/mychardev`	Reads back exactly the string that was written	Pass
6	`rmmod mychardev`	`/dev/mychardev` is removed	Pass
7	`modprobe mychardev`	Reloads successfully, `/dev/mychardev` reappears	Pass

Test results

The results of executing the test commands on the virtual machine are presented in Listing 4.3.

Listing 4.3: Driver test results on QEMU

```
root@qemux86-64:~# lsmod | grep mychardev
mychardev              12288  0
root@qemux86-64:~# modinfo mychardev
filename:       /lib/modules/6.6.151-yocto-standard/updates/mychardev.ko
version:        0.1
description:    Simple character device driver for Yocto/QEMU
author:         ngan
root@qemux86-64:~# ls -l /dev/mychardev
crw-rw-rw-    1 root     root       10, 259 Oct 11 11:33 /dev/mychardev
root@qemux86-64:~# echo "Xin chao tu Yocto!" > /dev/mychardev
root@qemux86-64:~# cat /dev/mychardev
Xin chao tu Yocto!
root@qemux86-64:~# rmmod mychardev
root@qemux86-64:~# ls /dev/mychardev
ls: /dev/mychardev: No such file or directory
root@qemux86-64:~# modprobe mychardev
root@qemux86-64:~# ls -l /dev/mychardev
crw-rw-rw-    1 root     root       10, 259 Oct 11 11:33 /dev/mychardev
```

The kernel log records exactly the order in which the driver's functions were called (Listing 4.4).

Listing 4.4: Kernel log (`dmesg`) during testing

```
root@qemux86-64:~# dmesg | grep mychardev
[    1.905815] mychardev: loading out-of-tree module taints kernel.
[    1.908313] mychardev: loaded, minor=259
[    9.504767] mychardev: opened
[    9.505949] mychardev: wrote 19 bytes
[    9.507121] mychardev: closed
[    9.509484] mychardev: opened
[    9.510464] mychardev: closed
[    9.514783] mychardev: unloaded
[    9.521525] mychardev: loaded, minor=259
```

Analysis of the results

Module loading: `lsmod` shows that the module occupies 12,288 bytes of memory with a usage count of 0. `modinfo` confirms that the module is installed in the correct

`updates/` location reserved for out-of-tree modules.

Device file: the leading character `c` confirms that this is a character device; the number pair (10, 259) shows that the device belongs to the misc class (major 10) with a dynamically allocated minor 259; the permissions `rw-rw-rw-` correspond to `.mode = 0666`.

Data write: the kernel recorded 19 bytes, exactly the 18 characters of the string `"Xin chao tu Yocto!"` plus the newline character added by `echo`. The sequence `opened – wrote – closed` correctly reflects the call order of `open`, `write`, `release`.

Data read: `cat` returns exactly the written string and stops when `read` returns 0 (EOF).

Removal and reload: after `rmmod`, the device file disappears, showing that the exit function released its resources correctly. `modprobe` reloads the module successfully, showing that `depmod` was executed correctly when the package was installed.

Limitations of the driver

During testing, several limitations of the current driver were identified:

No handling of the `O_TRUNC` flag: the variable `dev_len` only grows and never shrinks; when a short string is written after a long one, the tail of the old string remains. This can be fixed by setting `dev_len = 0` in the `open` function when `file->f_flags & O_TRUNC`.

Writing beyond the buffer capacity: when the write position reaches 1024 bytes, `simple_write_to_buffer()` returns 0, which may cause the user program to repeat the write operation. The error code `-ENOSPC` should be returned in this case.

Static, fixed-size buffer: dynamic allocation and configuring the size through a module parameter are not yet supported.

The system has only been tested on the emulation platform and has not yet been deployed to real hardware.

4.4 Overall Evaluation

Table 4.10 compares the achieved results with the objectives and research questions set out in Chapter 1.

Table 4.10: Objectives versus results

Objective	**Result achieved**	**Status**
Question 1: custom Yocto build for the Raspberry Pi 4	Built `core-image-base` (5,806 tasks), 329 MB WIC image, boots successfully with `systemd`, with UART debugging	Completed
Question 2: boot architecture of the BCM2711	Detailed description of the 7-stage GPU-first boot process, with a table of boot files	Completed
Question 3: native C versus Python	Equivalent latency (about 160 ms), C reduces RSS memory by 67.7% to 69.6% (10.12 vs. 33.72 MiB), 100% matching predictions	Completed
Question 4: multi-core acceleration, system bottleneck	3.67× speedup with 4 threads, throughput of 21.55 images/s; inference accounts for 93.30% to 98.32% of the time	Completed
QEMU emulation and device driver	Built for `qemux86-64`; the `mychardev` driver loads automatically at boot and passes all 7 test cases	Completed

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The project successfully deployed the quantized MobileNet V1 model on the Raspberry Pi 4 using Yocto Linux and TensorFlow Lite, with two inference engines in Python and C; it also built a QEMU emulation branch and a device driver integrated into the system image. The main results are:

Operating system: a customized Yocto Linux distribution, built from source with Poky Scarthgap and `meta-raspberrypi`, integrating `systemd` and hardware UART debugging and optimizing disk usage with `rm_work`, producing a bootable 329 MB WIC disk image.

Boot architecture: a complete, systematic description of the 7-stage boot process of the Raspberry Pi 4, characterized by the GPU-first model of the VideoCore VI, loading the bootloader from the SPI EEPROM, handing over to the Cortex-A72 cores and finally the Linux kernel mounting the root filesystem.

AI stack: the `meta-edge-ai` layer adds recipes that fix the missing C API headers in the upstream `meta-tensorflow-lite`, packaging both `libtensorflowlite_c.so` and `tflite_runtime` for Python.

Native C versus Python: the inference latencies are almost identical (about 160 ms with 1 thread), but native C reduces peak RSS memory by 67.7% to 69.6% (10.12 MiB versus 33.72 MiB), with 100% matching predictions.

Multi-core acceleration: increasing from 1 to 4 threads reduces latency from about 161 ms to about 43 ms (a 3.67$\times$ speedup, a reduction of about 72.8%), and throughput reaches 21.55 images/s, exceeding the real-time threshold.

Large-scale benchmark: on 1,000 Imagenette images, all configurations produced identical results (0 discrepancies), with a Top-1 accuracy of 69.10%; multi-threading does not change the classification results.

Bottleneck: the inference step accounts for 93.30% to 98.32% of the processing time of each image; the `performance` mode of the CPU governor brings no significant improvement over `ondemand` but raises the temperature by an additional 8.8 to 19.0 °C.

QEMU emulation and device driver: the `core-image-minimal` image (kernel 6.6.151) was successfully built for `qemux86-64`; the character driver `mychardev` is packaged with a recipe inheriting the `module` class, loads automatically at boot, creates `/dev/mychardev` and works correctly in all 7 test cases (read, write, removal and reload).

The results show that the performance of AI execution on embedded systems can be optimized without changing the structure or the weights of the model. The entire system is described by configuration files and recipes, which makes it highly reproducible.

5.2 Limitations and Future Work

Limitations

The measurements used tensors that had been preprocessed on Ubuntu, and do not yet include the time needed to decode and resize JPEG images directly on the Raspberry Pi.

The number of independent repetitions is still limited, and power consumption has not been evaluated.

The `mychardev` driver is a virtual device that stores data in kernel memory and has only been tested on QEMU; it does not yet control a physical peripheral and has not been deployed to the Raspberry Pi. The driver also still has the limitations noted in Section 4.3 (the `O_TRUNC` flag, writes beyond the buffer, the static buffer).

Future work

Improve the measurements: include the image decoding and resizing steps in the measured path, increase the number of independent repetitions and add power measurements.

Optimize boot time: measure the boot time with `systemd-analyze` and shorten it by disabling unnecessary services (`systemctl mask`).

Improve the driver: fix the limitations noted above, allocate the buffer dynamically, and add `ioctl`, `sysfs`/`procfs` attributes and `poll`/`select` support.

Bring the driver to real hardware: write a platform driver combined with the Device Tree for a real peripheral on the Raspberry Pi 4 (e.g., GPIO) and package it in a dedicated layer.

ARM emulation: use `qemuarm`/`qemuarm64` to test the same image and driver on an architecture that matches the real board before flashing the SD card.

REFERENCES

[1] A. Gherzan, *meta-raspberrypi: Yocto Project BSP layer for the Raspberry Pi boards (Scarthgap branch)*, 2024. [Online]. Available: `https://github.com/agherzan/meta-raspberrypi.git` (download source of the BSP layer for the Raspberry Pi 4). Accessed: Oct. 13, 2026.

[2] The Yocto Project, *Poky: The Yocto Project Reference System (Scarthgap branch)*, 2024. [Online]. Available: `https://git.yoctoproject.org/poky` (download source of the Poky source code). Accessed: Oct. 13, 2026.

[3] The Yocto Project, *The Yocto Project Documentation and Reference Manuals*, 2024. [Online]. Available: `https://docs.yoctoproject.org`. Accessed: Oct. 13, 2026.

[4] OpenEmbedded, *OpenEmbedded: The Build Framework for Embedded Linux*, 2024. [Online]. Available: `https://www.openembedded.org/`. Accessed: Oct. 13, 2026.

[5] Google LLC, *Grace Hopper test image for the TensorFlow Lite label image example*, 2024. [Online]. Available: `https://raw.githubusercontent.com/tensorflow/tensorflow/master/tensorflow/lite/examples/label_image/testdata/grace_hopper.bmp`. Accessed: Oct. 13, 2026.

[6] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017.

[7] Google LLC, *TensorFlow Lite – On-device machine learning*. [Online]. Available: `https://www.tensorflow.org/lite`

[8] R. Purdie, C. Larson, P. Blundell, *BitBake User Manual*, 2024. [Online]. Available: `https://docs.yoctoproject.org/bitbake/`

[9] J. Corbet, A. Rubini, G. Kroah-Hartman, *Linux Device Drivers*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2005.

[10] The Linux Kernel Documentation, *Building External Modules*. [Online]. Available: `https://docs.kernel.org/kbuild/modules.html`

[11] The Linux Kernel Documentation, *Misc devices*. [Online]. Available: `https://docs.kernel.org/driver-api/misc_devices.html`

[12] QEMU Project, *QEMU System Emulation User's Guide*. [Online]. Available: `https://www.qemu.org/docs/master/system/`

APPENDIX A

LINUX COMMAND REFERENCE TABLE

This appendix lists the Linux commands used while carrying out the project, grouped by function. Groups 1 to 6 relate to the Edge AI part on the Raspberry Pi 4; group 7 relates to the QEMU emulation and device driver part.

Table A.1: Reference table of the Linux commands used in the project

Command syntax and example	Technical purpose and role in the project
1. Package management & host environment	
`sudo apt update`	Updates the list of the latest software packages from Ubuntu's repositories.
`sudo apt install -y <package_list>`	Automatically installs the tools required to build Yocto (gcc, g++, diffstat, texinfo, chrpath, zstd, cpio...). The `-y` option automatically confirms the installation without asking again.
`sudo locale-gen en_US.UTF-8`	Generates the standard UTF-8 character encoding (Yocto requires the host system to run a standard UTF-8 locale to avoid text parsing errors).
`df -h /`	Checks the remaining free disk space (`-h` is human-readable, shown in GB/MB). Used to monitor the disk usage of the virtual machine.
`ip a`	Shows the list of network interfaces and the current IP addresses of the Ubuntu virtual machine.
`sudo dhclient -v`	Asks the DHCP server to reassign a dynamic IP address to the virtual machine's network interface when the network connection is lost.
`ping -c 2 google.com`	Sends 2 test packets (ICMP Echo) to Google's server to confirm that the Internet connection works.
2. Source code management & Yocto	
`git clone -b <branch> <URL> <directory>`	Downloads source code from GitHub/GitLab on the required branch (`-b scarthgap`). Used to clone Poky and the BSP layers (`meta-raspberrypi`).
`cd <directory>`	Changes the current working directory (Change Directory).
`ls -lh <path>`	Lists files in detail (`-l`: permissions/owner details, `-h`: file sizes shown in readable KB, MB).
`source oe-init-build-env [directory]`	Loads the Yocto environment initialization script (sets the `PATH` and `BBPATH` variables for the current shell and moves into the `build/` directory).
`bitbake-layers add-layer <path>`	Registers a new layer in the configuration file `conf/bblayers.conf`.

Command syntax and example	Technical purpose and role in the project
`bitbake-layers show-layers`	Shows all layers enabled in the project together with their priorities.
`bitbake-layers show-recipes`	Lists all recipes (`.bb`) that BitBake recognizes in the layers.
`bitbake-layers show-appends`	Checks whether the append files (`.bbappend`) are correctly linked to their original recipes.
`nano <file> / vim <file>`	Command-line text editors, used to edit configuration files such as `local.conf`.
`cat <file>`	Reads the entire contents of a text file and prints it to the terminal.
`head -n <lines> / tail -n <lines>`	Quickly views the first lines (`head`) or the last lines (`tail`) of a text file or log file.
3. System compilation (BitBake)	
`bitbake core-image-base`	Starts the Yocto build process: fetches the source code, cross-compiles the toolchain, compiles the kernel and packages the Linux image for the Raspberry Pi.
`bitbake benchmark-c`	Compiles only the C program with Yocto's cross-compiler, according to the recipe that was written.
`bitbake -c cleansstate <package>`	Thoroughly cleans the cache and the work directory of a specific package after a failure so that BitBake rebuilds it from scratch.
`oe-pkgdata-util list-pkgs`	Queries Yocto's database to check which software packages have been packaged successfully.
4. Image handling & virtual partitions	
`bzcat <file.wic.bz2> > <file.wic>`	Decompresses a Bzip2 disk image into a `.wic` file while keeping the original compressed file.
`lsblk`	Lists the block storage devices (hard disks, memory cards, loop devices) as a partition tree.
`sudo losetup -Pf <file.wic>`	Attaches the `.wic` disk image to a virtual loop device (`loop10`); the `-P` option automatically scans and splits the sub-partitions (`loop10p1`, `loop10p2`).
`sudo mkdir -p <directory>`	Creates a new directory (the `-p` option also creates the parent directories if they do not exist).
`sudo mount <partition> <directory>`	Mounts a disk partition into the Linux directory tree in order to access and view its data.
`sudo umount <directory_or_partition>`	Safely unmounts a disk partition, flushing all buffered data from RAM to the disk before disconnecting.
`sudo losetup -d <loop_device>`	Detaches and releases the virtual loop device after the inspection is finished.
`sudo dd if=<file> of=<device> bs=4M status=progress conv=fsync`	Raw block-level copy used to flash the `.wic` image onto the SD card (`if`: source file, `of`: target device, `bs`: 4 MB block size, `conv=fsync`: forces synchronization to the memory chip).
`sync`	Forces the Linux kernel to write all data still held in the RAM buffers to the physical card before it is removed from the computer.
5. Hardware debugging (UART) & kit checks	
`ls /dev/ttyUSB*`	Checks whether the computer has recognized the USB-to-UART adapter (CP2102/CH340).

Command syntax and example	Technical purpose and role in the project
`sudo picocom -b 115200 /dev/ttyUSB0`	Opens a serial console terminal to the Raspberry Pi 4 over the COM port at the standard baud rate of 115,200.
`uname -a`	Reads the identification information of the Linux operating system (host name, kernel version 6.6.63-v7l, ARMv7l chip architecture).
`cat /proc/cpuinfo`	Reads the specifications of the hardware processor managed by Linux (Broadcom BCM2711 chip, 4 CPU cores).
`free -h`	Checks the physical RAM of the Raspberry Pi 4 kit (total capacity, RAM in use and free RAM).
`cat .../scaling_governor`	Reads the current CPU clock frequency governor mode (`ondemand` or `performance`).
6. AI measurement & evaluation	
`sha256sum <file>`	Computes the SHA-256 cryptographic hash to verify integrity and ensure that the AI model/dataset file has not been altered.
`wc -c <file> / wc -l <file>`	Counts the total number of bytes (`-c`) or lines of text (`-l`) of a file.
`gzip -t <file.gz>`	Checks the integrity of a Gzip file without decompressing it to disk.
`grep -iE '<keyword>' <file>`	Searches for and filters lines containing a keyword (case-insensitive `-i`, extended regular expressions `-E`).
`find <directory> -name '<pattern>'`	Searches for files or subdirectories by name across the entire directory tree.
`python3 -c "<code>"`	Quickly runs a Python snippet directly from the terminal without creating a `.py` script file.
`./<program>`	Runs a compiled binary executable in the current directory (e.g., `./benchmark_c`).
`systemd-analyze`	Tool for measuring the total boot time of the operating system (kernel loading time and user-space loading time).
`systemctl mask <service>`	Permanently masks a systemd process/service to prevent it from starting and slowing down the system.
7. QEMU emulation and device driver	
`bitbake-layers create-layer <path>`	Creates the skeleton of a new layer (here `meta-mydriver`) following Yocto conventions.
`bitbake mychardev`	Builds only the recipe of the `mychardev` driver (compiles the module and packages it).
`runqemu qemux86-64 nographic slirp kvm`	Boots the newly built image on QEMU: no graphical window, user-mode networking (no `sudo` required), KVM acceleration enabled.
`ssh -p 2222 root@127.0.0.1`	Accesses the QEMU virtual machine via SSH (port 2222 of the host machine is forwarded to port 22 of the virtual machine).
`lsmod | grep mychardev`	Checks whether the `mychardev` module has been loaded into the kernel.
`modinfo mychardev`	Shows module information: file name, version, description, author.
`modprobe mychardev / rmmod mychardev`	Loads (with dependency handling) or removes the module from the kernel while the system is running.

Command syntax and example	Technical purpose and role in the project
`dmesg | grep mychardev`	Shows the kernel log messages written by the driver (`loaded`, `opened`, `wrote`, `closed`, `unloaded`).
`echo "hello" > /dev/mychardev; cat /dev/mychardev`	Writes data to the character device and reads it back to test the read/write functionality of the driver.

APPENDIX B

SOURCE CODE OF THE `MYCHARDEV` DRIVER

B.1 Driver source code `mychardev.c`

```c
// SPDX-License-Identifier: GPL-2.0
/*
 * mychardev - simple character device driver example.
 * Registers /dev/mychardev (misc device). Data written to it is stored
 * in a kernel buffer and can be read back.
 */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/miscdevice.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/mutex.h>

#define DEVICE_NAME "mychardev"
#define BUF_SIZE    1024

static char dev_buf[BUF_SIZE];
static size_t dev_len;
static DEFINE_MUTEX(dev_lock);

static int mychardev_open(struct inode *inode, struct file *file)
{
        pr_info(DEVICE_NAME ": opened\n");
        return 0;
}

static int mychardev_release(struct inode *inode, struct file *file)
{
        pr_info(DEVICE_NAME ": closed\n");
        return 0;
}

static ssize_t mychardev_read(struct file *file, char __user *ubuf,
                              size_t count, loff_t *ppos)
{
        ssize_t ret;

        mutex_lock(&dev_lock);
        ret = simple_read_from_buffer(ubuf, count, ppos, dev_buf, dev_len);
        mutex_unlock(&dev_lock);
        return ret;
}

static ssize_t mychardev_write(struct file *file, const char __user *ubuf,
                               size_t count, loff_t *ppos)
{
        ssize_t ret;

        mutex_lock(&dev_lock);
        ret = simple_write_to_buffer(dev_buf, BUF_SIZE, ppos, ubuf, count);
        if (ret > 0 && *ppos > dev_len)
                dev_len = *ppos;
        mutex_unlock(&dev_lock);
        if (ret > 0)
                pr_info(DEVICE_NAME ": wrote %zd bytes\n", ret);
        return ret;
}
```

```
58
59 static const struct file_operations mychardev_fops = {
60         .owner   = THIS_MODULE,
61         .open    = mychardev_open,
62         .release = mychardev_release,
63         .read    = mychardev_read,
64         .write   = mychardev_write,
65         .llseek  = default_llseek,
66 };
67
68 static struct miscdevice mychardev_misc = {
69         .minor = MISC_DYNAMIC_MINOR,
70         .name  = DEVICE_NAME,
71         .fops  = &mychardev_fops,
72         .mode  = 0666,
73 };
74
75 static int __init mychardev_init(void)
76 {
77         int ret = misc_register(&mychardev_misc);
78
79         if (ret) {
80                 pr_err(DEVICE_NAME ": misc_register failed: %d\n", ret);
81                 return ret;
82         }
83         pr_info(DEVICE_NAME ": loaded, minor=%d\n", mychardev_misc.minor);
84         return 0;
85 }
86
87 static void __exit mychardev_exit(void)
88 {
89         misc_deregister(&mychardev_misc);
90         pr_info(DEVICE_NAME ": unloaded\n");
91 }
92
93 module_init(mychardev_init);
94 module_exit(mychardev_exit);
95
96 MODULE_LICENSE("GPL");
97 MODULE_AUTHOR("ngan");
98 MODULE_DESCRIPTION("Simple character device driver for Yocto/QEMU");
99 MODULE_VERSION("0.1");
```

B.2 Makefile

```
1 obj-m := mychardev.o
2
3 SRC := $(shell pwd)
4
5 all:
6         $(MAKE) -C $(KERNEL_SRC) M=$(SRC)
7
8 modules_install:
9         $(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules_install
10
11 clean:
12         rm -f *.o *~ core .depend .*.cmd *.ko *.mod.c *.mod
13         rm -f Module.markers Module.symvers modules.order
14         rm -rf .tmp_versions Modules.symvers
```

B.3 Recipe `mychardev_0.1.bb`

```
SUMMARY = "Simple character device driver (/dev/mychardev)"
DESCRIPTION = "Example out-of-tree Linux kernel module built with Yocto"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://COPYING;md5=12f884d2ae1ff87c09e5b7ccc2c4ca7e"

inherit module

SRC_URI = "file://Makefile \
           file://mychardev.c \
           file://COPYING \
          "

S = "${WORKDIR}"

RPROVIDES:${PN} += "kernel-module-mychardev"

# Load the module automatically at boot
KERNEL_MODULE_AUTOLOAD += "mychardev"
```